

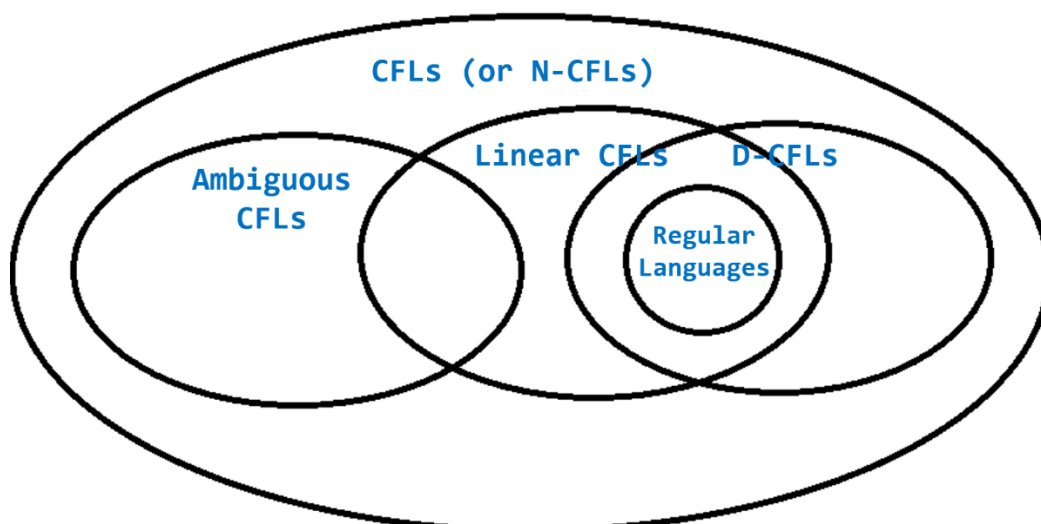
Ambiguous CFGs and Inherently ambiguous CFLs

Ambiguous Grammar

- A context free grammar is called ambiguous if there exists **more than one leftmost derivation (LMD)** or **more than one rightmost derivation (RMD)** for a string which is generated by grammar. **Or**
- A context free grammar is called ambiguous if there exists more than one parse tree for a string which is generated by grammar. **Or**
- A CFG is ambiguous if one or more terminal strings have multiple leftmost derivations from the start symbol. Equivalently: multiple rightmost derivations, or multiple parse trees.

Note:

- 1) **An ambiguous grammar is a context free grammar (CFG)** for which there exists a string that can have more than one leftmost derivation or rightmost derivation or parse tree.
- 2) Every non-empty context-free language (CFL) admits an ambiguous grammar by introducing a duplicate production rule.
- 3) **A language that only admits ambiguous grammars** is called an **inherently ambiguous language**.
- 4) **Deterministic context-free grammars (DCFGs)** are always **unambiguous**, and are an important subclass of unambiguous grammars; there are non-deterministic unambiguous grammars, however.



Important Notes:

Note-1:

If N parse trees exist for any string w of grammar G , then there will be

- N -corresponding leftmost derivations and
- N -corresponding rightmost derivations.

Note-2:

For ambiguous grammars, more than one leftmost derivation and more than one rightmost derivation exist for at least one string.

Leftmost derivation and rightmost derivation may be same or different.

Note-3:

For unambiguous grammars, a unique leftmost derivation and a unique rightmost derivation exist for all the strings.

Leftmost derivation and rightmost derivation represent the same parse tree.

Example: Consider the unambiguous grammar

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{num} \mid \text{id}$

Let us consider a string $w = \text{id} + \text{id} * \text{id}$

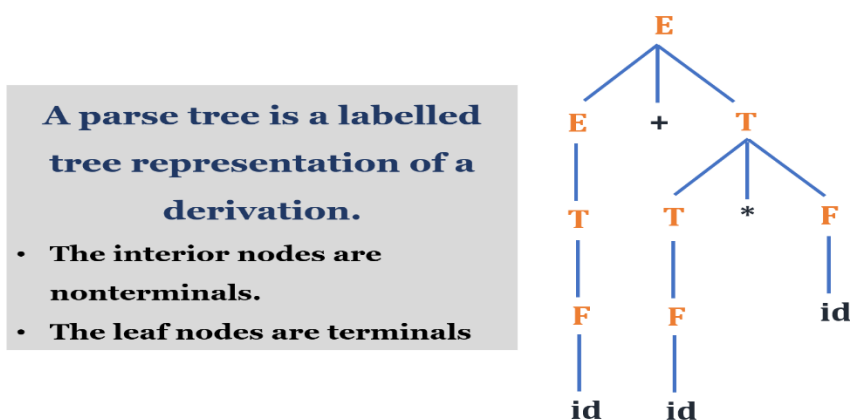
Leftmost derivation

$E \Rightarrow E + T$
 $\Rightarrow T + T$
 $\Rightarrow F + T$
 $\Rightarrow \text{id} + T$
 $\Rightarrow \text{id} + T * F$
 $\Rightarrow \text{id} + F * F$
 $\Rightarrow \text{id} + \text{id} * F$
 $\Rightarrow \text{id} + \text{id} * \text{id}$

Rightmost derivation

$E \Rightarrow E + T$
 $\Rightarrow E + T * F$
 $\Rightarrow E + T * \text{id}$
 $\Rightarrow E + F * \text{id}$
 $\Rightarrow E + \text{id} * \text{id}$
 $\Rightarrow T + \text{id} * \text{id}$
 $\Rightarrow F + \text{id} * \text{id}$
 $\Rightarrow \text{id} + \text{id} * \text{id}$

The Parse tree for string $\text{id} + \text{id} * \text{id}$ is



For both the Leftmost derivation and Rightmost derivation, the parse tree is the same.

Note: For unambiguous grammars, it is not possible to get two different parse trees for the same input string either using Leftmost derivations and/or using Rightmost derivations.

Note-4:

Leftmost derivation and rightmost derivation of a string is exactly same in some grammars (i.e., in linear grammars).

In fact, there may exist a grammar in which leftmost derivation and rightmost derivation is exactly same for all the strings.

Example: Consider the linear grammar

$$S \rightarrow aS \mid \epsilon$$

The language generated by this grammar is $L = \{ a^n \mid n \geq 0 \}$ and Regex = a^*

All the strings generated from this grammar have their leftmost derivation and rightmost derivation exactly same.

Let us consider a string $w = aaa$.

Leftmost Derivation:

$S \Rightarrow aS$
 $\Rightarrow aaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaa$ (Using $S \rightarrow \epsilon$)

Rightmost Derivation:

$S \Rightarrow aS$
 $\Rightarrow aaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaa$ (Using $S \rightarrow \epsilon$)

Clearly, **Leftmost derivation = Rightmost derivation**

Similar is the case for all other strings.

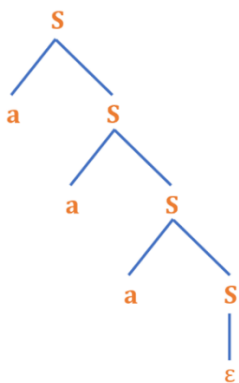
Note-5:

For a given parse tree, we may have its leftmost derivation exactly same as rightmost derivation.

Example: Consider the linear grammar

$$S \rightarrow aS \mid \epsilon$$

Let us consider a parse tree for the string 'aaa'



Leftmost Derivation:

$S \Rightarrow aS$
 $\Rightarrow aaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaa$ (Using $S \rightarrow \epsilon$)

Rightmost Derivation:

$S \Rightarrow aS$
 $\Rightarrow aaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaaS$ (Using $S \rightarrow aS$)
 $\Rightarrow aaa$ (Using $S \rightarrow \epsilon$)

Clearly, for a given parse tree, the leftmost derivation is exactly same as the rightmost derivation.

Note-6:

If for all the strings of a grammar (i.e., in linear grammars), leftmost derivation is exactly same as rightmost derivation, then that grammar may be ambiguous or unambiguous.

Note-7:

There may exist derivations for a string which are neither leftmost nor rightmost.

Example:

Consider the following grammar

$$S \rightarrow ABC$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$C \rightarrow c$$

Consider a string $w = abc$

Total 6 derivations exist for string w .

The following 3 derivations are neither leftmost nor rightmost

Derivation-01:

$S \Rightarrow ABC$
 $\Rightarrow aBC$ (Using $A \rightarrow a$)
 $\Rightarrow aBc$ (Using $C \rightarrow c$)
 $\Rightarrow \mathbf{abc}$ (Using $B \rightarrow b$)

Derivation-02:

$S \Rightarrow ABC$
 $\Rightarrow AbC$ (Using $B \rightarrow b$)
 $\Rightarrow abC$ (Using $A \rightarrow a$)
 $\Rightarrow \mathbf{abc}$ (Using $C \rightarrow c$)

Derivation-03:

$S \Rightarrow ABC$
 $\Rightarrow AbC$ (Using $B \rightarrow b$)
 $\Rightarrow Abc$ (Using $C \rightarrow c$)
 $\Rightarrow \mathbf{abc}$ (Using $A \rightarrow a$)

Ambiguous Grammar Examples:

Example 1: $S \rightarrow S \mid \epsilon$

The simplest example is the above ambiguous grammar for the trivial language that consists of only the empty string.

$S \rightarrow S \mid \epsilon$

meaning that the nonterminal S can be derived to either itself again, or to the empty string. Thus, the empty string has leftmost (or rightmost) derivations of length 1, 2, 3, and indeed of any length, depending on how many times the rule $S \rightarrow S$ is used.

Consider a string $w = \epsilon$

First Leftmost derivation

$S \Rightarrow \epsilon$

Second Leftmost derivation

$S \Rightarrow S \Rightarrow S \Rightarrow S \Rightarrow \epsilon$

This language also has an unambiguous grammar, consisting of a single **production rule**:

$S \rightarrow \epsilon$

meaning that the unique production can produce only the empty string, which is the unique string in the language.

In the same way, **any grammar for a non-empty language can be made ambiguous by adding duplicates.**

Example 2: Consider the below grammars:

1) $S \rightarrow aS \mid a \mid \epsilon$

2) $S \rightarrow Sa \mid a \mid \epsilon$

3) $S \rightarrow Sa \mid aS \mid \epsilon$

These are all examples of ambiguous grammars for the language $L = \{ a^n \mid n \geq 0 \}$

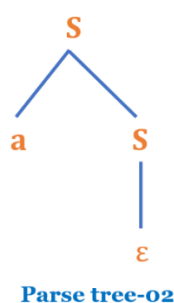
Consider a string 'a' for grammar $S \rightarrow aS \mid a \mid \epsilon$

First Leftmost derivation

$S \Rightarrow a$

Second Leftmost derivation

$S \Rightarrow aS$
 $\Rightarrow a$

Parse trees:

Since two different LMDs and two different parse trees exist for the string 'a', so the grammar is ambiguous.

The language $L = \{a^n \mid n \geq 0\}$ of the above grammar also has an unambiguous grammar:

$S \rightarrow aS \mid \epsilon$ **or** $S \rightarrow Sa \mid \epsilon$

Here, each string has unique leftmost derivation and unique rightmost derivation.

Why Ambiguity in Grammar is not suitable for compiler construction?

- Ambiguity in grammar is not suitable for compiler construction because it can make it difficult for a compiler to determine which parse tree is correct for a given input string. This is because ambiguous grammar can produce multiple parse trees for the same input string.
- Ambiguous grammar is not parsed by any parsers.

Recognizing ambiguous grammars

1) How to find out whether the given grammar is ambiguous?

There is no algorithm for detecting whether an arbitrary grammar is ambiguous.

Here are some ways to determine if a grammar is ambiguous:

- **Check for multiple derivations**
If a string can be derived from more than one left most derivation or more than one right most derivation, then the grammar is ambiguous.
- **Check for multiple parse trees**
If an input string can have more than one parse tree, then the grammar is ambiguous

2) How to convert ambiguous grammar to unambiguous grammar?

- There is no algorithm for converting an arbitrary ambiguous grammar into an unambiguous one.
- If the grammar has ambiguity, no method can automatically detect and remove the ambiguity, but we can remove ambiguity by re-writing the whole grammar without ambiguity.
- Ambiguous grammar (that generates arithmetic expressions / boolean expressions / regular expressions having operators) can be transformed into unambiguous grammar by using some methods like the definition of operator precedence and associativity, by which only one interpretation will be correct.

Ambiguity removal in Context Free Grammars

Example 1: Arithmetic Expression Grammar

Here is a CFG that generates arithmetic expressions (having addition, subtraction, multiplication, division and exponentiation operators).

$E \rightarrow E + E$
 $E \rightarrow E - E$
 $E \rightarrow E * E$
 $E \rightarrow E / E$
 $E \rightarrow E ^ E$
 $E \rightarrow (E) \mid \text{num} \mid \text{id}$

What these production rules tell us is that the result of any operation, for example, multiplication, is also an expression (denoted, E).

From the above grammar, string "**id+id-id**" can be derived in two ways using Leftmost derivations:

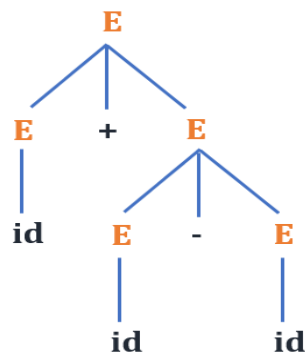
First Leftmost derivation

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E - E$
 $\Rightarrow id + id - E$
 $\Rightarrow id + id - id$

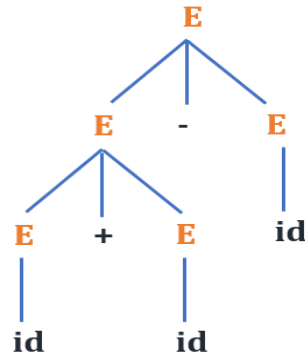
Second Leftmost derivation

$E \Rightarrow E - E$
 $\Rightarrow E + E - E$
 $\Rightarrow id + E - E$
 $\Rightarrow id + id - E$
 $\Rightarrow id + id - id$

Parse Trees



Parse tree-01



Parse tree-02

From the above grammar, string "**id+id-id**" can be derived in two ways using Rightmost derivations:

First Rightmost derivation

$E \Rightarrow E + E$
 $\Rightarrow E + E - E$
 $\Rightarrow E + E - id$
 $\Rightarrow E + id - id$
 $\Rightarrow id + id - id$

Second Rightmost derivation

$E \Rightarrow E - E$
 $\Rightarrow E - id$
 $\Rightarrow E + E - id$
 $\Rightarrow E + id - id$
 $\Rightarrow id + id - id$

Since there are two leftmost derivations/two parse trees/two rightmost derivations for a string "**id+id-id**", the above CFG is ambiguous.

Removal of Ambiguity in arithmetic expression grammar:

We can remove ambiguity solely on the basis of the following two properties:

1. Precedence

If different operators are used, we will consider the precedence of the operators. The three important characteristics are:

- 1) **The level at which the production is present denotes the priority of the operator used.**
- 2) The **production at higher levels** will have **operators with less priority**.
In the parse tree, the nodes which are at top levels or close to the root node will contain the lower priority operators.
- 3) The **production at lower levels** will have **operators with higher priority**.
In the parse tree, the nodes which are at lower levels or close to the leaf nodes will contain the higher priority operators.

2. Associativity

If the same precedence operators are in production, then we will have to consider the associativity.

- **If the associativity is left to right, then we have to prompt a left recursion in the production.** The parse tree will also be left recursive and grow on the left side.
+, -, *, / are left associative operators.
- **If the associativity is right to left, then we have to prompt the right recursion in the production.** The parse tree will also be right recursive and grow on the right side.
^ is a right associative operator.

Arithmetic expression grammar after removing ambiguity

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow E - T \end{aligned} \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{ (rules are Left recursive because + and - have left association)}$$

$$E \rightarrow T$$

$$\begin{aligned} T &\rightarrow T * F \\ T &\rightarrow T / F \end{aligned} \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{ (rules are Left recursive because * and / have left association)}$$

$$T \rightarrow F$$

$$F \rightarrow D \wedge F \quad \text{(rule is right recursive because the exponentiation operator is right associative)}$$

$$F \rightarrow D$$

$$D \rightarrow (E) \mid \text{num} \mid \text{id}$$

Note:

- E - Expression, T - Term, F - Factor, num - number and id - identifier
- In math expressions, terms are the components added or subtracted, factors are the elements multiplied within each term, and coefficients are the numbers multiplying variables.

Example 2: Conditional Boolean expressions grammar

Ambiguous grammar:

$$B \rightarrow B \text{ or } B \mid B \text{ and } B \mid \text{not } B \mid E \text{ relop } E \mid \text{true} \mid \text{false}$$

From the above grammar String 'true and false or true' can be derived in two ways:

First Leftmost derivation

$$\begin{aligned} B &\Rightarrow B \text{ or } B \\ &\Rightarrow B \text{ and } B \text{ or } B \\ &\Rightarrow \text{true and } B \text{ or } B \\ &\Rightarrow \text{true and false or true} \\ &\Rightarrow \text{true and false or true} \end{aligned}$$

Second Leftmost derivation

$$\begin{aligned} B &\Rightarrow B \text{ and } B \\ &\Rightarrow \text{true and } B \\ &\Rightarrow \text{true and } B \text{ or } B \\ &\Rightarrow \text{true and false or } B \\ &\Rightarrow \text{true and false or true} \end{aligned}$$

Since there are two leftmost derivations for a string 'true and false or true', the above CFG is ambiguous.

Unambiguous grammar: (Ambiguity is removed by considering operator precedence)

$$\begin{aligned} B &\rightarrow B \text{ or } C \mid C \\ C &\rightarrow C \text{ and } D \mid D \\ D &\rightarrow E \text{ relop } E \mid F \\ F &\rightarrow \text{not } F \mid G \\ G &\rightarrow (B) \mid \text{true} \mid \text{false} \end{aligned}$$

Note: E represents arithmetic expression, *relop* is relational operator, *or* is logical OR, *and* is logical AND.

Example 3: Regular Expression grammar

Ambiguous grammar:

$$R \rightarrow R+R \mid RR \mid R^* \mid R^+ \mid R? \mid (R) \mid a \mid b \mid c$$

From the above grammar string 'a+bc' can be derived in two ways:

First Leftmost derivation

$R \Rightarrow R + R$
 $\Rightarrow a + R$
 $\Rightarrow a + RR$
 $\Rightarrow a + bR$
 $\Rightarrow a + bc$

Second Leftmost derivation

$R \Rightarrow RR$
 $\Rightarrow R + RR$
 $\Rightarrow a + RR$
 $\Rightarrow a + bR$
 $\Rightarrow a + bc$

Since there are two leftmost derivations for a string 'a+bc', the above CFG is ambiguous.

Unambiguous grammar: (Ambiguity is removed by considering operator precedence)

$R \rightarrow R+S \mid S$ [Note: + in this rule represents alternation or Union]
 $S \rightarrow ST \mid T$
 $T \rightarrow T^* \mid T^+ \mid T? \mid U$ [Note: + in this rule represents Kleene plus]
 $U \rightarrow (R) \mid a \mid b \mid c$

Only one Leftmost derivation for the string 'a+bc':

$R \Rightarrow R + S$
 $\Rightarrow S + S$
 $\Rightarrow T + S$
 $\Rightarrow U + S$
 $\Rightarrow a + S$
 $\Rightarrow a + ST$
 $\Rightarrow a + TT$
 $\Rightarrow a + UT$
 $\Rightarrow a + bT$
 $\Rightarrow a + bU$
 $\Rightarrow a + bc$

Example 4: Dangling else

A common example of ambiguity in computer programming languages is the dangling else problem.

In many languages, **the else in an if-then(-else) statement is optional**, which results in nested conditionals having multiple ways of being recognized in terms of the context-free grammar.

Conditionals in two valid forms:

- **if-then form** and
- **if-then-else form**

In a grammar containing the rules

$S \rightarrow \text{if } C \text{ then } S$
 $S \rightarrow \text{if } C \text{ then } S \text{ else } S$
 $S \rightarrow \dots$ [other statements rule]
 $C \rightarrow \dots$ [boolean expressions rule]

some ambiguous phrase structures can appear. The expression

if a then if b then S1 else S2

can be parsed as either

if a then begin if b then S1 end else S2

or as

if a then begin if b then S1 else S2 end

depending on whether the else is associated with the first if or second if.

This is resolved in various ways in different languages.

- Sometimes the grammar is modified so that it is unambiguous, such as
 - by **requiring an endif statement** or
 - by **making else mandatory**

Unambiguous grammar for if-then-else statements

$S \rightarrow \text{matched-stmt} \mid \text{open-stmt}$

$\text{matched-stmt} \rightarrow \text{if } C \text{ then matched-stmt else matched-stmt} \mid \dots [\text{other statements}]$

$\text{open-stmt} \rightarrow \text{if } C \text{ then } S \mid \text{if } C \text{ then matched-stmt else open-stmt}$

- In other cases, the grammar is left ambiguous, but the ambiguity is resolved by making the overall phrase grammar context-sensitive, such as by associating an else with the nearest if. In this case the phrase grammar is unambiguous, but the context-free grammar is ambiguous.

Exercise:

1) Find out whether the following grammars are ambiguous or not?

- a) $S \rightarrow aSbS \mid bSaS \mid \epsilon$ [generates set of strings where the number of a's equals the number of b's]
- b) $S \rightarrow 1S \mid 11S \mid \epsilon$ [generates 1^*]
- c) $S \rightarrow AB \mid aB$ [generates $(a)^+ b$]
 $A \rightarrow a \mid aA$
 $B \rightarrow b$

Solutions: The following strings have multiple LMDs or parse trees

- a) baba or abab
- b) 111
- c) ab

Inherently ambiguous grammar

- Inherently ambiguous grammar is a context-free grammar (CFG) that cannot be made unambiguous, and the language it generates is also inherently ambiguous.
- Inherently ambiguous grammar is a CFG that can't be made unambiguous, and is used to describe a language that can't be generated by an unambiguous context-free grammar.

Note: A CFL L is inherently ambiguous if every CFG for L is ambiguous.

Examples

- Consider the languages $L_1 = \{ a^n b^n c^m \mid n \geq 0 \text{ and } m \geq 1 \}$ and $L_2 = \{ a^n b^m c^m \mid m \geq 0 \text{ and } n \geq 1 \}$

The grammar of language $L = L_1 \cup L_2$ is an example of an inherently ambiguous grammar.

$S \rightarrow A \mid B$

$A \rightarrow CD$

$C \rightarrow aCb \mid \epsilon$ [generates $a^n b^n \mid n \geq 0$]

$D \rightarrow cD \mid c$ [generates $c^m \mid m \geq 1$]

$B \rightarrow EF$

$E \rightarrow aE \mid a$ [generates $a^n \mid n \geq 1$]

$F \rightarrow bFc \mid \epsilon$ [generates $b^m c^m \mid m \geq 0$]

or

$S \rightarrow A \mid B$

$A \rightarrow Ac \mid c \mid C$

$C \rightarrow aCb \mid \epsilon$

$B \rightarrow aB \mid a \mid D$

$D \rightarrow bDc \mid \epsilon$

Strings that belong to both the languages are $L_1 \cap L_2 = \{ a^n b^n c^n \mid n \geq 1 \}$.

Example string: abc

First Leftmost derivation

$S \Rightarrow A$
 $\Rightarrow CD$
 $\Rightarrow aCbD$
 $\Rightarrow abD$
 $\Rightarrow \text{abc}$

Second Leftmost derivation

$S \Rightarrow B$
 $\Rightarrow EF$
 $\Rightarrow aF$
 $\Rightarrow abFc$
 $\Rightarrow \text{abc}$

For every string $a^n b^n c^n$ where $n \geq 1$, there exists two parse trees either using LMDs or RMDs. **Hence, the grammar is ambiguous.**

This CFG can't be made unambiguous.

Inherently ambiguous languages

- An inherently ambiguous language is a context-free language that cannot be recognized by any unambiguous grammar.
In other words, some languages are inherently ambiguous, meaning that no unambiguous grammar exists for them.
- While some context-free languages (the set of strings that can be generated by a grammar) have both ambiguous and unambiguous grammars, **there exist context-free languages for which no unambiguous context-free grammar can exist. Such languages are called inherently ambiguous.**
- **A Context Free Language L is inherently ambiguous if every CFG for Context Free Language L is ambiguous.**

Proving ambiguity:

- Proving that a language is inherently ambiguous is difficult and undecidable in general (i.e., for complex context-free languages).
- However, there are some methods that can be used to prove the ambiguity of some simple languages, including:
 - Iterations on derivation trees
 - Iterations on semi-linear sets
 - Generating series

Note: There are no inherently ambiguous regular languages.

Examples:

1) Is the Language $L = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and either } i = j \text{ or } j = k \}$ inherently ambiguous?

Solution: Is the grammar of L ambiguous? Why or why not?

Idea: This language is simply the union of $L_1 = \{ a^i b^j c^k \mid i, j, k \geq 0, i = j \}$ and $L_2 = \{ a^i b^j c^k \mid i, j, k \geq 0, j = k \}$.

We can create simple grammars for the separate languages and union them:

$S \rightarrow A \mid B$

For L_1 , we simply ensure that the number of a's equals the number of b's:

$A \rightarrow Ac \mid C \mid \epsilon$

$C \rightarrow aCb \mid \epsilon$

Similarly, for ensuring that the number of b's equals the number of c's:

$B \rightarrow aB \mid D \mid \epsilon$

$$D \rightarrow bDc \mid \epsilon$$

This grammar is ambiguous. For string $w = a^n b^n c^n$, we may use either A or B production rule to generate w.

2) Find out whether the given grammar is inherently ambiguous or not.

$$S \rightarrow aaS \mid aaaS \mid \epsilon$$

Solution:

First, we will prove that the grammar is ambiguous by showing that a string 'aaaaa' has more than one LMDs

First Leftmost derivation

$$\begin{aligned} S &\Rightarrow aaS \\ &\Rightarrow aaaaaS \\ &\Rightarrow aaaaa \end{aligned}$$

Second Leftmost derivation

$$\begin{aligned} S &\Rightarrow aaaS \\ &\Rightarrow aaaaaS \\ &\Rightarrow aaaaa \end{aligned}$$

Two LMDs for the string "aaaaa" indicates that the grammar is ambiguous.

Try constructing another grammar which is unambiguous.

$$\text{Regex} = (aa + aaa)^* \text{ or } (\epsilon + aaa^*)$$

$$\text{Language } L = \{ \epsilon, aa, aaa, aaaa, aaaaa, \dots \}$$

Grammar:

$$\begin{array}{l|l} S \rightarrow aaA \mid \epsilon & \text{or} \quad S \rightarrow aA \mid \epsilon \\ A \rightarrow aA \mid \epsilon & A \rightarrow aA \mid a \end{array}$$

This grammar is **unambiguous**.

Hence, the given grammar is **not inherently ambiguous**.

Note:

- First prove that the grammar is ambiguous by deriving multiple LMDs or RMDs. Then try constructing another grammar which is unambiguous (if possible).

2) Find out whether the given grammar is inherently ambiguous or not.

$$S \rightarrow SbS \mid a$$

$$\text{Solution: } L = \{ a, aba, ababa, abababa, \dots \} \quad \text{Regex} = a(ba)^* \text{ or } (ab)^*a$$

First, we prove that the grammar is ambiguous by showing that a string has multiple LMDs.

Input string = **ababa**

First Leftmost derivation

$$\begin{aligned} S &\Rightarrow SbS \\ &\Rightarrow SbSbS \\ &\Rightarrow abSbS \\ &\Rightarrow ababS \\ &\Rightarrow ababa \end{aligned}$$

Second Leftmost derivation

$$\begin{aligned} S &\Rightarrow SbS \\ &\Rightarrow abS \\ &\Rightarrow abSbS \\ &\Rightarrow ababS \\ &\Rightarrow ababa \end{aligned}$$

Two LMDs for the string "ababa" indicates that the grammar is ambiguous.

Try constructing another grammar which is unambiguous.

$$\begin{array}{l|l|l|l} S \rightarrow Aa & \text{or} \quad S \rightarrow abS \mid a & \text{or} \quad S \rightarrow aB & \text{or} \quad S \rightarrow Sba \mid a \\ A \rightarrow Aab \mid \epsilon & & B \rightarrow baB \mid \epsilon & \end{array}$$

These grammars are **unambiguous**.

Hence, the given grammar is **not inherently ambiguous**.

Exercise:

- 1) Find out whether the given grammar is inherently ambiguous or not.
 $S \rightarrow AB \mid aaaB$
 $A \rightarrow Aa \mid a$
 $B \rightarrow b$
- 2) Find out whether $L = \{ \{a^n b^n c^m d^m \mid n, m > 0\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\} \}$ is inherently ambiguous or not.

Note:

The language L of our grammar G_1 (below) is not inherently ambiguous, even though the grammar G_1 is ambiguous, but there exist some grammars (i.e., G_2) that are not ambiguous for language L .

Ambiguous Grammar G_1 :

$$S \rightarrow AS \mid \epsilon$$
$$A \rightarrow A1 \mid 0A1 \mid 01$$

Consider a string '00111':

LMD1: $S \Rightarrow AS \Rightarrow 0A1S \Rightarrow 0A11S \Rightarrow 00111S \Rightarrow 00111$

LMD2: $S \Rightarrow AS \Rightarrow A1S \Rightarrow 0A11S \Rightarrow 00111S \Rightarrow 00111$

Two LMDs for the string "00111" indicates that the grammar is ambiguous.

Change the ambiguous grammar to force the extra 1's to be generated last.

Unambiguous Grammar G_2 :

$$S \rightarrow AS \mid \epsilon$$
$$A \rightarrow 0A1 \mid B$$
$$B \rightarrow B1 \mid 01$$

$S \Rightarrow AS \Rightarrow 0A1S \Rightarrow 0B1S \Rightarrow 0B11S \Rightarrow 00111$

Only one LMD for the string "00111". Hence the grammar G_2 is Unambiguous.

Why to Care ambiguity?

- Ambiguity of the grammar implies that at least some strings in its language have different structures (parse trees).
 - Thus, such a grammar is unlikely to be useful for a programming language, because two structures for the same string (program) implies two different meanings (executable equivalent programs) for this program.
 - Common example: the easiest grammars for arithmetic expressions are ambiguous and need to be replaced by more complex, unambiguous grammars.
- An inherently ambiguous language would be absolutely unsuitable as a programming language, because we would not have any way of fixing a unique structure for all its programs.

Note:

1. If a context free grammar G is ambiguous, language generated by grammar $L(G)$ may or may not be ambiguous.
2. It is not always possible to convert ambiguous CFG to unambiguous CFG. Only some ambiguous CFG can be converted to unambiguous CFG.
3. There is no algorithm to convert ambiguous CFG to unambiguous CFG.
4. There always exists a unambiguous CFG corresponding to unambiguous CFL.
5. Deterministic CFL are always unambiguous and are parsed by LR parsers.

Questions:

1) Consider the following statements about the context free grammar

$$S \rightarrow SS \mid ab \mid ba \mid \epsilon$$

- I. G is ambiguous
- II. G produces all strings with equal number of a's and b's
- III. G can be accepted by a deterministic PDA

Which combination below expresses all the true statements about G?

- a) I only
- b) I and III only
- c) II and III only
- d) I, II and III

Solution:

I. There are different LMD's for string abab which can be

LMD1: $S \Rightarrow \underline{S}S \Rightarrow \underline{S}SS \Rightarrow ab\underline{S}S \Rightarrow abab\underline{S} \Rightarrow abab$

LMD2: $S \Rightarrow \underline{S}S \Rightarrow ab\underline{S} \Rightarrow abab$

Even ϵ also has infinite number of derivations.

So the grammar is ambiguous. So, statement I is true.

II. "G produces all strings with equal number of a's and b's", is false since $L(G) = (ab+ba)^*$ and this language cannot produce all strings with equal number of a's and b's. For example aabb has equal number of a's and b's but $aabb \notin L(G)$.

III. "G can be accepted by a DPDA" is true, since $L(G) = (ab+ba)^*$ and this is a regular language, since we have written a regular expression for it. Since every regular language is also a DCFL, a DPDA exists. which accepts this language.

2) Which one of the following statements is FALSE?

- A. There exist context-free languages such that all the context-free grammars generating them are ambiguous.
- B. An unambiguous context free grammar always has a unique parse tree for each string of the language generated by it.
- C. Both deterministic and non-deterministic pushdown automata always accept the same set of languages.
- D. A finite set of string from one alphabet is always a regular language.

Solution:

(A) is correct because for ambiguous CFL's, all CFG corresponding to it are ambiguous.

(B) is also correct as unambiguous CFG has a unique parse tree for each string of the language generated by it.

(C) is false as some languages are accepted by Non – deterministic PDA but not by deterministic PDA.

(D) is also true as finite set of string is always regular.

So option (C) is correct option.

Ambiguity is a common feature of natural languages, where it is tolerated and dealt with in a variety of ways. In programming languages, where there should be only one interpretation of each statement, ambiguity must be removed when possible. Often, we can achieve this by rewriting the grammar in an equivalent, unambiguous form.

Note: CFG Relationship with other Computation Models

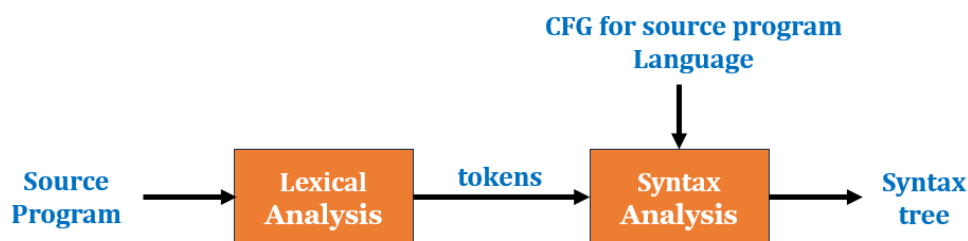
- A context-free grammar can be generated by pushdown automata just as regular languages can be generated by finite state machines. Since **all regular languages can be generated by CFGs, all regular languages can too be generated by pushdown automata.**
- **Any language that can be generated using regular expressions can be generated by a context-free grammar.**
- The way to do this is to take the regular language, determine its finite state machine and write production rules that follow the transition functions.

Note: Operators: Order of Precedence

Type	Operator	Precedence Level
Parentheses	()	High ↑ ↓ Low
Unary Operator	! ~ -	
Multiplication/ Division/Modulus	* / %	
Addition/ Subtraction	+ -	
Shift	<< >>	
Comparison	< <= > >=	
Equality	== !=	
Bitwise AND	&	
Bitwise XOR	^	
Bitwise OR		
Logical AND	&&	
Logical OR		

Chomsky Normal Form (CNF)

- A Context-Free Grammar (CFG) is especially useful in the field of both computer science and linguistics for describing the structure of programming languages and natural languages.
- Context-Free Grammars (CFGs) are powerful enough to express the syntax of most programming languages.
- **Syntax analyzer** (also known as Parser) receives the source code in the form of tokens from the lexical analyzer and determines whether the input token stream is syntactically well formed (i.e., by constructing a parse tree) with respect to a given grammar and creates an output a tree-like intermediate representation (i.e., syntax tree).



- Depending upon how the parse tree is built, parsing techniques are classified into three general categories:
 1. Universal,
 2. Top-down, and
 3. Bottom-up.
- Universal parsing methods such as the Cocke-Younger-Kasami (CYK) algorithm and Earley's algorithm can parse any grammar. These general methods are, however, too inefficient to use in production compilers.
- In computer science, the Cocke-Younger-Kasami algorithm (alternatively called CYK, or CKY) is a parsing algorithm for context-free grammars published by Itiroo Sakai in 1961.
- **The standard version of CYK operates only on context-free grammars (CFGs) given in Chomsky normal form (CNF).**
- The problem is that context-free grammars can be very complicated, with long and complex rules. It is therefore desirable to be able to describe the original CFG in a simplified ("normal") form.
- If G is a CFG in CNF, then for any string $w \in L(G)$ of length $n \geq 1$, exactly $2n-1$ steps are required for any derivation of w .

Chomsky Normal Form (CNF)

The first such simplified form was produced by Noam Chomsky, the famous linguist and the author of many concepts actively used in programming languages.

CNF Definition:

Noam Chomsky showed that every CFG can be transformed into a simplified form, in which only three types of rules are allowed:

- $A \rightarrow BC$, where A, B, and C are variables.
- $A \rightarrow a$, where A is a variable and a is a terminal symbol; and
- $S \rightarrow \epsilon$, where S is the start symbol of the grammar, and ϵ means an empty string. This rule can only appear if ϵ is in $L(G)$.

A context-free grammar, G , is said to be in Chomsky normal form if all of its production rules are of the above forms.

What is the application of Chomsky's Normal Form?

- CNF is used to make the grammar more structured and easier to analyze.
- A CFG in CNF is used in **natural language parsing (NLP)**, **compiler design**, **grammar optimization** etc.

Why Chomsky Normal Form?

- **The key advantage is that in CNF, every derivation of a string of n letters has exactly $2n-1$ steps. Thus, one can determine if a string is in the language by exhaustive search of all derivations.**
- Normal forms make many concepts much easier to handle because you can assume a simple structure for them.
- CNF enables a polynomial time algorithm to decide whether a string can be generated by the grammar.

CFG to CNF conversion steps:

Step 1: Eliminate lambda (or epsilon) productions from non-starting variables

Determine the nullable variables (those that generate empty string (ϵ), i.e., X is nullable, if there is a derivation if $X \xRightarrow{*} \epsilon$). Go through all productions, and for each, omit every possible subset of nullable variables.

- For example, if G is

$$\begin{array}{l} P \rightarrow AxB \\ A \rightarrow \epsilon \\ B \rightarrow \epsilon \end{array} \quad \Rightarrow \quad P \rightarrow x$$

(Here, both A and B is deriving only epsilon or lambda, then consider only without A and B in production body), After eliminating all epsilon or lambda productions, the CFG is $P \rightarrow x$

- For example, if G is

$$\begin{array}{l} P \rightarrow AxB \\ A \rightarrow a \mid \epsilon \\ B \rightarrow \epsilon \end{array} \quad \Rightarrow \quad \begin{array}{l} P \rightarrow Ax \\ A \rightarrow a \mid \epsilon \end{array} \quad \Rightarrow \quad \begin{array}{l} P \rightarrow Ax \mid x \\ A \rightarrow a \end{array}$$

After eliminating lambda production $B \rightarrow \epsilon$ (Here, B is deriving only epsilon or lambda, then consider only without B in production body), the CFG is $P \rightarrow Ax$ and $A \rightarrow a \mid \epsilon$

After eliminating lambda production, $A \rightarrow \epsilon$ (Here, A is deriving not only epsilon or lambda, but also derives a, then consider with and without A in production body), the CFG is $P \rightarrow Ax \mid x$ and $A \rightarrow a$

- For example, if G is

$$\begin{array}{lcl} P \rightarrow AxB & & P \rightarrow AxB \mid xB \\ A \rightarrow a \mid \epsilon & \Rightarrow & A \rightarrow a \\ B \rightarrow b \mid \epsilon & & B \rightarrow b \end{array} \quad \Rightarrow \quad \begin{array}{l} P \rightarrow AxB \mid Ax \mid xB \mid x \\ A \rightarrow a \\ B \rightarrow b \end{array}$$

After eliminating epsilon or lambda production $A \rightarrow \epsilon$, (Here, A is deriving not only epsilon or lambda, but also derives a, then consider with and without A in production body), the CFG is $P \rightarrow AxB \mid xB$ and $A \rightarrow a, B \rightarrow b \mid \epsilon$

After eliminating epsilon or lambda production $B \rightarrow \epsilon$, (Here, B is deriving not only epsilon or lambda, but also derives b, then consider with and without B in production body), the CFG is $P \rightarrow AxB \mid Ax \mid xB \mid x$ and $A \rightarrow a, B \rightarrow b$

Step2: Eliminate Unit productions of type Variable→Variable (i.e., of type $A \rightarrow B$)

- Consider production $A \rightarrow B$. Then for every production $B \rightarrow \alpha$, add the production $A \rightarrow \alpha$. Repeat until done (but don't re-create a unit production already deleted).
- For example, if G is

$$\begin{array}{lcl} S \rightarrow aXbX \mid abX \mid aXb \mid ab & & S \rightarrow aXbX \mid abX \mid aXb \mid ab \\ X \rightarrow aY \mid bY \mid a \mid b & \Rightarrow & X \rightarrow aY \mid bY \mid a \mid b \\ Y \rightarrow X & & Y \rightarrow aY \mid bY \mid a \mid b \\ Y \rightarrow c & & Y \rightarrow c \end{array}$$

Step 3: Eliminate Useless Symbols (Eliminate non-generating and non-reachable symbols)

There are 2 aspects:

- Derivability:** Each Variable must derive a string/must end with set of terminal(s).
For example, If we drop production rules containing non-generating B from the following grammar

$$\begin{array}{lcl} S \rightarrow AB \mid a & \text{we obtain} & S \rightarrow a \quad (\text{i.e., after dropping the production rule } S \rightarrow AB) \\ A \rightarrow b & & A \rightarrow b \quad (\text{Now A is unreachable and became useless}) \end{array}$$

- Reachability:** Each Variable must be reachable from S.
For example, if we drop production rules containing unreachable C and c from the following grammar

$$\begin{array}{lcl} S \rightarrow Aa \mid a & \text{we obtain} & S \rightarrow Aa \mid a \\ A \rightarrow b & & A \rightarrow b \\ C \rightarrow c & & \end{array}$$

Step 4: Convert the CFG to CNF as per definition.

- Move Terminals to Unit Productions of type Variable→terminal**

That is, for every terminal on the RHS of a non-unit production, add a substitute variable.

For example, replacing b by B in RHS and by introducing new Rule $B \rightarrow b$ in the following grammar

$$\begin{array}{lcl} A \rightarrow bC \mid bb \mid a \mid b & \text{we obtain} & A \rightarrow BC \mid BB \mid a \mid b \\ C \rightarrow c & & C \rightarrow c \\ & & B \rightarrow b \end{array}$$

- Replace Long Production bodies (i.e., length>2) by Shorter Ones**
(i.e., Eliminate right-hand sides with more than 2 nonterminals)

For example, in the following grammar

$A \rightarrow BCD$ then after replacing long production bodies, we obtain

$$\begin{array}{lcl} A \rightarrow BE & \text{or} & A \rightarrow ED \\ E \rightarrow CD & & E \rightarrow BC \end{array}$$

Note-1:

1. The CNF normalization is performed in order to standardize the grammar, because the RHS of grammar productions have no specific format and hence the parsing process is hard.
2. The CNF idea is to make every step in derivation useful.
3. For a given grammar, there can be more than one CNF.
4. CNF produces the same language as generated by CFG.
5. Chomsky Normal Form (CNF) turns every non-binary parse tree into binary parse tree.
6. The different order of CNF conversion steps (i.e., 2, 1, 3, 4 or 4, 2, 1, 3 ...) may lead to an exponential blow up in the number of rules in some CFGs.

Note-2: Useless symbols

Consider a grammar $G = (N, T, P, S)$

$X \in N \cup T$ is **useful**, if there is a derivation $S \xRightarrow{*} \alpha X \beta \xRightarrow{*} w$ for $w \in T^*$

$X \in N \cup T$ is **useless**, if X is not useful.

It is clear that eliminating production rules involving useless symbols have no impact on the language of the grammar.

Note-3: Generating and reachable symbols

Consider a grammar $G = (N, T, P, S)$

$X \in N \cup T$ is **generating**, if $X \xRightarrow{*} w$ for some $w \in T^*$

$X \in N \cup T$ is **reachable**, if there is some derivation $S \xRightarrow{*} \alpha X \beta$

Example 1: Convert the following CFG into an equivalent CFG in CNF.

$S \rightarrow aXbX$

$X \rightarrow aY \mid bY \mid \epsilon$

$Y \rightarrow X \mid c$

Solution:

Step 1: Eliminate lambda productions

The variable X is nullable; and so therefore is Y . After elimination of ϵ , we obtain:

$S \rightarrow aXbX \mid abX \mid aXb \mid ab$

$X \rightarrow aY \mid bY \mid a \mid b$

$Y \rightarrow X \mid c$

Step2: Eliminate Unit productions of type Variable $\rightarrow$ Variable

After elimination of the unit production $Y \rightarrow X$, we obtain:

$S \rightarrow aXbX \mid abX \mid aXb \mid ab$

$X \rightarrow aY \mid bY \mid a \mid b$

$Y \rightarrow aY \mid bY \mid a \mid b \mid c$

Step 3: Eliminating Useless productions and Symbols

No useless productions and Symbols.

Step 4: Convert the CFG to CNF as per definition.

Move Terminals to Unit Productions of type **Variable**→**terminal**

(i.e., For every terminal on the RHS of a non-unit production, add a substitute variable.)

Replace a by A, and b by B, by introducing new Rules $A \rightarrow a$ and $B \rightarrow b$

$S \rightarrow AXBX \mid ABX \mid AXB \mid AB$

$X \rightarrow AY \mid BY \mid a \mid b$

$Y \rightarrow AY \mid BY \mid a \mid b \mid c$

$A \rightarrow a$

$B \rightarrow b$

Replace Long Production bodies (i.e., length>2) by Shorter Ones

$S \rightarrow EF \mid AF \mid EB \mid AB$

$X \rightarrow AY \mid BY \mid a \mid b$

$Y \rightarrow AY \mid BY \mid a \mid b \mid c$

$E \rightarrow AX$

$F \rightarrow BX$

$A \rightarrow a$

$B \rightarrow b$

Example 2: Convert the following CFG into an equivalent CFG in CNF.

$S \rightarrow AbA$

$A \rightarrow Aa \mid \epsilon$

Solution:

Step 1: Eliminate lambda productions

$S \rightarrow AbA$
 $A \rightarrow Aa \mid \epsilon$ $\Rightarrow$ $S \rightarrow AbA \mid bA \mid Ab \mid b$
 $A \rightarrow Aa \mid a$

Step2: Eliminate Unit productions of type **Variable**→**Variable**

No unit productions

Step 3: Eliminating Useless productions and Symbols

No useless productions and Symbols.

Step 4: Convert the CFG to CNF as per definition.

Move Terminals to Unit Productions of type **Variable**→**terminal**

$S \rightarrow AbA \mid bA \mid Ab \mid b$
 $A \rightarrow Aa \mid a$ $\Rightarrow$ $S \rightarrow ABA \mid BA \mid AB \mid b$
 $A \rightarrow AC \mid a$

$B \rightarrow b$

$C \rightarrow a$

Replace Long Productions by Shorter Ones

$S \rightarrow ABA \mid BA \mid AB \mid b$

$A \rightarrow AC \mid a$

$B \rightarrow b$

$C \rightarrow a$

$S \rightarrow TA \mid BA \mid AB \mid b$

$T \rightarrow AB$

$A \rightarrow AC \mid a$

$B \rightarrow b$

$C \rightarrow a$

Note: If G is a CFG in CNF, then for any string $w \in L(G)$ of length $n \geq 1$, exactly $2n-1$ steps are required for any derivation of w.

Example: Consider the string 'aba' and the above CFG in CNF.

LMD: $S \Rightarrow TA$

$\Rightarrow ABA$

$\Rightarrow aBA$

$\Rightarrow abA$

$\Rightarrow \mathbf{aba}$ The string 'aba' is derived in $2n-1=2*3-1=5$ steps.

Example 3: Convert the following CFG into an equivalent CFG in CNF.

$S \rightarrow AB$
 $A \rightarrow aAA \mid \epsilon$
 $B \rightarrow bBB \mid \epsilon$

Solution:

Step 1: Eliminate lambda productions

The variables A and B are both nullable; and so therefore is S.

$S \rightarrow AB$
 $A \rightarrow aAA \mid \epsilon$
 $B \rightarrow bBB \mid \epsilon$

$\Rightarrow$

$S \rightarrow AB \mid \epsilon \mid A \mid B$
 $A \rightarrow aAA \mid a \mid aA$
 $B \rightarrow bBB \mid b \mid bB$

Step2: Eliminate Unit productions of type Variable→Variable

$S \rightarrow AB \mid \epsilon \mid A \mid B$
 $A \rightarrow aAA \mid a \mid aA$
 $B \rightarrow bBB \mid b \mid bB$

$\Rightarrow$

$S \rightarrow AB \mid \epsilon \mid aAA \mid a \mid aA \mid bBB \mid b \mid bB$
 $A \rightarrow aAA \mid a \mid aA$
 $B \rightarrow bBB \mid b \mid bB$

Step 3: Eliminating Useless productions and Symbols

No useless productions and Symbols.

Step 4: Convert the CFG to CNF as per definition.

Move Terminals to Unit Productions of type Variable→terminal

$S \rightarrow AB \mid \epsilon \mid aAA \mid a \mid aA \mid bBB \mid b \mid bB$
 $A \rightarrow aAA \mid a \mid aA$
 $B \rightarrow bBB \mid b \mid bB$

$\Rightarrow$

$S \rightarrow AB \mid \epsilon \mid CAA \mid a \mid CA \mid DBB \mid b \mid DB$
 $A \rightarrow CAA \mid a \mid CA$
 $B \rightarrow DBB \mid b \mid BB$
 $C \rightarrow a$
 $D \rightarrow b$

Replace Long Productions by Shorter Ones

$S \rightarrow AB \mid \epsilon \mid CAA \mid a \mid CA \mid DBB \mid b \mid DB$
 $A \rightarrow CAA \mid a \mid CA$
 $B \rightarrow DBB \mid b \mid BB$
 $C \rightarrow a$
 $D \rightarrow b$

$\Rightarrow$

$S \rightarrow AB \mid \epsilon \mid EA \mid a \mid CA \mid FB \mid b \mid DB$
 $A \rightarrow EA \mid a \mid CA$
 $B \rightarrow FB \mid b \mid BB$
 $C \rightarrow a$
 $D \rightarrow b$
 $E \rightarrow CA$
 $F \rightarrow DB$

Example 4: Convert the following CFG into an equivalent CFG in CNF.

$S \rightarrow ASA \mid A \mid \epsilon$
 $A \rightarrow 00 \mid \epsilon$

Solution:

Step 1: Eliminate lambda productions

$S \rightarrow ASA \mid A \mid \epsilon$
 $A \rightarrow 00 \mid \epsilon$

$\Rightarrow$

$S \rightarrow ASA \mid S \mid SA \mid AS \mid A \mid \epsilon$
 $A \rightarrow 00$

Step2: Eliminate Unit productions of type Variable→Variable

Eliminate first, the unit production $S \rightarrow S$ because it's useless.

$S \rightarrow ASA \mid S \mid SA \mid AS \mid A \mid \epsilon$
 $A \rightarrow 00$

$\Rightarrow$

$S \rightarrow ASA \mid SA \mid AS \mid A \mid \epsilon$
 $A \rightarrow 00$

Eliminate next, the unit production $S \rightarrow A$

$S \rightarrow ASA \mid SA \mid AS \mid A \mid \epsilon$

$A \rightarrow \emptyset\emptyset$



$S \rightarrow ASA \mid SA \mid AS \mid \emptyset\emptyset \mid \epsilon$

$A \rightarrow \emptyset\emptyset$

Step 3: Eliminating Useless productions and Symbols

No useless productions and Symbols.

Step 4: Convert the CFG to CNF as per definition.

Move Terminals to Unit Productions of type **Variable**→**terminal**

$S \rightarrow ASA \mid SA \mid AS \mid \emptyset\emptyset \mid \epsilon$

$A \rightarrow \emptyset\emptyset$



$S \rightarrow ASA \mid SA \mid AS \mid BB \mid \epsilon$

$A \rightarrow BB$

$B \rightarrow \emptyset$

Replace Long Productions by Shorter Ones

$S \rightarrow ASA \mid SA \mid AS \mid BB \mid \epsilon$

$A \rightarrow BB$

$B \rightarrow \emptyset$



$S \rightarrow CA \mid SA \mid AS \mid BB \mid \epsilon$

$C \rightarrow AS$

$A \rightarrow BB$

$B \rightarrow \emptyset$

Example 5: Convert the following CFG into an equivalent CFG in CNF.

$S \rightarrow ABA$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bBc \mid \epsilon$

Note: nullable(G) = {A, B, S}

Example 6: Convert the following CFG into an equivalent CFG in CNF.

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{num} \mid \text{id}$

Exercises:

1) Convert the following grammar to Chomsky Normal Form (CNF).

$S \rightarrow X \mid XYa \mid XbX$

$X \rightarrow Xa \mid \lambda$

$Y \rightarrow Yb \mid YZ$

$Z \rightarrow ZY \mid ZX \mid bY$

2) If G is the grammar in CNF, then fill the following table

w	n= w	Length of Derivation (2n-1)	Max-depth of the parse tree	Min-depth of the parse tree
λ	0	1		
a_1	1	1		
$a_1 a_2$	2	3		
$a_1 a_2 a_3$	3	5		
$a_1 a_2 a_3 a_4$	4	7		
$a_1 a_2 a_3 a_4 a_5$	5	9		
$a_1 a_2 a_3 a_4 a_5 a_6$	6	11		

Note:

- If G is a CFG in CNF, then for any string $w \in L(G)$ of length $n \geq 1$, exactly $2n-1$ steps are required for any derivation of w.

- In the tree, the total number of edges from the root node to the leaf node in the longest path is known as "Depth of Tree".

3) Convert the following CFG to CNF.

$S \rightarrow aAa \mid bBb \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S \mid \lambda$

4) Design a CNF grammar for the set of strings of balanced parentheses.

Advantages of Chomsky Normal Form (CNF)

Chomsky Normal Form (CNF) has many advantages in formal language theory and algorithm parsing. Some of these advantages include:

- **Simple and Standardized grammar**
- **Efficient Parsing Algorithms**
- **Less Grammar Size**
- **Elimination of Empty String Rules**

Questions

1) What is Chomsky's normal form in TOC?

- In TOC **Chomsky Normal Form** is a special form of context-free grammar (CFG).
- The production rules of **Chomsky Normal Form** are limited to only **single terminal** and **two non-terminal symbols**. If empty string (ϵ) is in $L(G)$, then $S \rightarrow \epsilon$, is allowed, where S is the start symbol of the grammar.

2) What is Chomsky's theory called?

- **Chomsky's theory**, formed by **Noam Chomsky** is called **Linguistic Theory** or **Chomskyan Generative Grammar**.

3) What are the key elements of Chomsky's theory?

- The key elements of **Chomsky's** theory include **Universal grammar** (noun, pronoun, adverb etc.), **transformational rules** between grammars, **surface** and **deep** structure language acquisition and most importantly a person's **innate ability** to learn grammar and language.

4) What is the application of Chomsky's Normal Form?

- Chomsky's Normal Form is a type of context free grammar which is used to make the grammar more structured and easier to analyze.
- CNF is used in **natural language processing**, **algorithm parsing**, **compiler design**, **grammar optimization** etc.

Additional Info:

Chomsky reduced form

Another way to define the Chomsky normal form is:

A formal grammar is in **Chomsky reduced form** if all of its production rules are of the form:

- $A \rightarrow BC$, where A, B, and C are variables.

- $A \rightarrow a$, where A is a variable and a is a terminal symbol;

Only those context-free grammars which do not generate the empty string can be transformed into Chomsky reduced form.

Chomsky's normal form (Approach-2)

The following steps are used to convert context-free grammar into Chomsky's normal form.

1. Introduce a new start variable that produces the old start variable (i.e., The start variable should not appear on the right-hand side of any rule)
2. Remove the null productions from non-starting variables.
3. Remove the unit productions.
4. Eliminate Useless Symbols (i.e., Eliminate non-generating and non-reachable symbols)
5. Convert all rules such that the right-hand side has only one terminal or two variables. New variables can be introduced to convert the rules.



Department of Computer Science and Engineering

PES University, Bangalore, India

UE23CS242A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

CYK Parsing Algorithm

- The Cocke–Younger–Kasami algorithm (aka CYK, or CKY) is a parsing algorithm for context free grammars (CFGs) in Chomsky normal form (CNF).
- The algorithm is named after its discoverers: John Cocke, Daniel Younger, and Tadao Kasami. CYK employs bottom-up parsing and dynamic programming.
- CYK Algorithm is a membership algorithm of CFG. It is used to decide whether a given string belongs to the language of grammar or not.
- The dynamic programming algorithm requires the CFG to be rendered into CNF, because it tests for possibilities to split the current sequence into two smaller sequences.
- The importance of the CYK algorithm stems from its high efficiency in certain situations. Using big O notation, the worst case running time of CYK is $O(n^3 \cdot |G|)$, where n is the length of the parsed string and $|G|$ is the size of the grammar G (i.e., $|G|$ be the number of production rules in G) in CNF.

Deciding Membership Using CYK Algorithm

To decide the membership of any given string, we construct a triangular table where each row of the table corresponds to one particular length of sub strings.

- Bottom most row corresponds to strings of length 1.
- Second row from bottom corresponds to strings of length 2.
- Third row from bottom corresponds to strings of length 3.
- ...
- Top most row from bottom corresponds to the given string of length n .

Example:

For a given string w of length 4 units, triangular table looks like

$X_{1,4}$				⇒	Strings of length 4
$X_{1,3}$	$X_{2,4}$			⇒	Strings of length 3
$X_{1,2}$	$X_{2,3}$	$X_{3,4}$		⇒	Strings of length 2
$X_{1,1}$	$X_{2,2}$	$X_{3,3}$	$X_{4,4}$	⇒	Strings of length 1

Note:

- In $X_{i,j}$ where i is the substring start index and j is the substring end index.
- Each cell in the table $X_{i,j}$ is the set of variables (or non-terminals) that can produce the substring spanning from position i to j .

For a given string w of length 5 units, triangular table looks like

$X_{1,5}$				
$X_{1,4}$	$X_{2,5}$			
$X_{1,3}$	$X_{2,4}$	$X_{3,5}$		
$X_{1,2}$	$X_{2,3}$	$X_{3,4}$	$X_{4,5}$	
$X_{1,1}$	$X_{2,2}$	$X_{3,3}$	$X_{4,4}$	$X_{5,5}$
w_1	w_2	w_3	w_4	w_5

Looking for pairs to compute $X_{1,5}$ variables

$X_{1,5}$				
$X_{1,4}$	$X_{2,5}$			
$X_{1,3}$	$X_{2,4}$	$X_{3,5}$		
$X_{1,2}$	$X_{2,3}$	$X_{3,4}$	$X_{4,5}$	
$X_{1,1}$	$X_{2,2}$	$X_{3,3}$	$X_{4,4}$	$X_{5,5}$

Note: Take the corresponding arrow position variables in computing $X_{1,5}$ variables

i.e., $(X_{1,1} \cdot X_{2,5}) \cup (X_{1,2} \cdot X_{3,5}) \cup (X_{1,3} \cdot X_{4,5}) \cup (X_{1,4} \cdot X_{5,5})$

CYK Algorithm:

1. Construct a triangular table of the length of your given string.
2. Each row corresponds to the length of the substrings of the given word 'w':
 - The bottom row will have strings of length 1.
 - The second row from the bottom will have strings of length 2 and so on.
- Each cell $X_{i,j}$ in the table is the set of variables (or non-terminals) that can produce the substring spanning from position i to j .
3. The first row cells (i.e., $X_{i,i}$) can be computed directly from the grammar (i.e., finding the variables that derive strings of length one). The second-row and onwards cells from the bottom can be computed by using pairs of previously computed sets.

To compute $X_{i,j}$ follow the below steps:

- Perform the below computation and get the results
 $(X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \dots \cup (X_{i,j-1} \cdot X_{j,j})$
 - Look for variables (i.e., Non-terminals) that derive (or generate) the above computed results and assign the set of variables that derive the above computed results to $X_{i,j}$
- Note:** Sometimes not even a single variable can derive (or generate) the above computed results, in that case assign Φ (empty set) to $X_{i,j}$

In the equation, $X_{i,i}$ or $X_{i,j}$ refers to the set of variables derived from the production rules.

4. If the top row consists of the starting variable of the grammar, then the given string can be derived from it.

Note: Here, dot(.) operator is performing cartesian product operation on two sets.

Example 1: Given CFG in CNF notation

$S \rightarrow AB \mid BC$

$A \rightarrow BA \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB \mid a$

And string $w = \text{baaba}$

Question: Is baaba in $L(G)$?

Solution:

Construct a triangular table of the length of your given string.

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12}	X_{23}	X_{34}	X_{45}	
X_{11}	X_{22}	X_{33}	X_{44}	X_{55}
b	a	a	b	a

Filling 1st row: (i.e., cells representing strings of length 1)

$X_{(i,i)}$ is a set of variables A such that $A \rightarrow w_i$ is a production rule of grammar G , where w_i is part of the given string 'w' or the whole string.

$X_{1,1} = B$ (Variables that derive substring $w_1=b$)

$X_{2,2} = A, C$ (Variables that derive substring $w_2=a$)

$X_{3,3} = A, C$ (Variables that derive substring $w_3=a$)

$X_{4,4} = B$ (Variables that derive substring $w_4=b$)

$X_{5,5} = A, C$ (Variables that derive substring $w_5=a$)

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12}	X_{23}	X_{34}	X_{45}	
X_{11} B	X_{22} A, C	X_{33} A, C	X_{44} B	X_{55} A, C
b	a	a	b	a

Filling 2nd row: (i.e., cells representing strings of length 2)

$X_{i,j} = (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \dots \cup (X_{i,j-1} \cdot X_{j,j})$

$X_{1,2} = (X_{1,1} \cdot X_{2,2}) = (X_{1,1} \cdot X_{2,2}) = \{B\} \cdot \{A, C\} = \{BA, BC\}$

Look for production rules to generate BA or BC . There are two: S and A

$X_{1,2} = \{S, A\}$

$$X_{2,3} = (X_{1,i} \cdot X_{i+1,j}) = (X_{2,2} \cdot X_{3,3}) = \{A,C\} \cdot \{A,C\} = \{AA, AC, CA, CC\} = \{B\}$$

$$X_{3,4} = (X_{i,i} \cdot X_{i+1,j}) = (X_{3,3} \cdot X_{4,4}) = \{A,C\} \cdot \{B\} = \{AB, CB\} = \{S,C\}$$

$$X_{4,5} = (X_{i,i} \cdot X_{i+1,j}) = (X_{4,4} \cdot X_{5,5}) = \{B\} \cdot \{A,C\} = \{BA, BC\} = \{S, A\}$$

X_{15}				
X_{14}	X_{25}			
X_{13}	X_{24}	X_{35}		
X_{12} S, A	X_{23} B	X_{34} S, C	X_{45} S, A	
X_{11} B	X_{22} A, C	X_{33} A, C	X_{44} B	X_{55} A, C
b	a	a	b	a

Filling 3rd row: (i.e., cells representing strings of length 3)

$$\begin{aligned}
 X_{1,3} &= (X_{1,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
 &= (X_{1,1} \cdot X_{2,3}) \cup (X_{1,2} \cdot X_{3,3}) \\
 &= \{B\} \cdot \{B\} \cup \{S, A\} \cdot \{A, C\} \\
 &= \{BB\} \cup \{SA, SC, AA, AC\} \\
 &= \{BB, SA, SC, AA, AC\}
 \end{aligned}$$

Look for production rules to generate BB, SA, SC, AA, AC. There are NONE

$$= \emptyset$$

$$\begin{aligned}
 X_{2,4} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
 &= (X_{2,2} \cdot X_{3,4}) \cup (X_{2,3} \cdot X_{4,4}) \\
 &= \{A,C\} \cdot \{S,C\} \cup \{B\} \cdot \{B\} \\
 &= \{AS, AC, CS, CC\} \cup \{BB\} \\
 &= B \cup \emptyset \\
 &= B
 \end{aligned}$$

$$\begin{aligned}
 X_{3,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
 &= (X_{3,3} \cdot X_{4,5}) \cup (X_{3,4} \cdot X_{5,5}) \\
 &= \{A,C\} \cdot \{S,A\} \cup \{S,C\} \cdot \{A,C\} \\
 &= \{AS, AA, CS, CA\} \cup \{SA, SC, CA, CC\} \\
 &= \emptyset \cup B \\
 &= B
 \end{aligned}$$

X_{15}				
X_{14}	X_{25}			
X_{13} Φ	X_{24} B	X_{35} B		
X_{12} S, A	X_{23} B	X_{34} S, C	X_{45} S, A	
X_{11} B	X_{22} A, C	X_{33} A, C	X_{44} B	X_{55} A, C
b	a	a	b	a

Filling 4th row: (i.e., cells representing strings of length 4)

$$\begin{aligned}
X_{1,4} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \\
&= (X_{1,1} \cdot X_{2,4}) \cup (X_{1,2} \cdot X_{3,4}) \cup (X_{1,3} \cdot X_{4,4}) \\
&= \{B\} \cdot \{B\} \cup \{S,A\} \cdot \{S,C\} \cup \Phi \cdot \{B\} \\
&= \{BB\} \cup \{SS, SC, AS, AC\} \cup \Phi \\
&= \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{2,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \\
&= (X_{2,2} \cdot X_{3,5}) \cup (X_{2,3} \cdot X_{4,5}) \cup (X_{2,4} \cdot X_{5,5}) \\
&= \{A,C\} \cdot \{B\} \cup \{B\} \cdot \{S,A\} \cup B \cdot \{A,C\} \\
&= \{AB, CB\} \cup \{BS, BA\} \cup \{BA, BC\} \\
&= \{S,C\} \cup A \cup \{A,S\} \\
&= \{S,A,C\}
\end{aligned}$$

X_{15}				
X_{14} Φ	X_{25} S, A, C			
X_{13} Φ	X_{24} B	X_{35} B		
X_{12} S, A	X_{23} B	X_{34} S, C	X_{45} S, A	
X_{11} B	X_{22} A, C	X_{33} A, C	X_{44} B	X_{55} A, C
b	a	a	b	a

Filling 5th row: (i.e., cells representing strings of length 5)

$$\begin{aligned}
X_{1,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \\
&= (X_{1,1} \cdot X_{2,5}) \cup (X_{1,2} \cdot X_{3,5}) \cup (X_{1,3} \cdot X_{4,5}) \cup (X_{1,4} \cdot X_{5,5}) \\
&= \{B\} \cdot \{S,A,C\} \cup \{S,A\} \cdot \{B\} \cup \Phi \cdot \{S,A\} \cup \Phi \cdot \{A,C\} \\
&= \{BS, BA, BC\} \cup \{SB, AB\} \cup \Phi \cup \Phi \\
&= \{A, S\} \cup \{S, C\} \\
&= \{S,A,C\}
\end{aligned}$$

X_{15} S, A, C				
X_{14} Φ	X_{25} S, A, C			
X_{13} Φ	X_{24} B	X_{35} B		
X_{12} S, A	X_{23} B	X_{34} S, C	X_{45} S, A	
X_{11} B	X_{22} A, C	X_{33} A, C	X_{44} B	X_{55} A, C
b	a	a	b	a

Is baaba in $L(G)$?

Yes

We can see the table, the cell X_{15} contains Non-terminals (S, A, C)

if $S \in X_{15}$ then $baaba \in L(G)$

Example 2: Show the CYK parsing with the Given grammar G in CNF notation.

$S \rightarrow NPVP$

$VP \rightarrow VPP_P \mid VN_P \mid \text{eats}$

$PP \rightarrow PN_P$

$NP \rightarrow DN \mid \text{she}$

$V \rightarrow \text{eats}$

$P \rightarrow \text{with}$

$N \rightarrow \text{fish} \mid \text{fork}$

$D \rightarrow \text{a}$

And string/sentence $w = \text{she eats a fish with a fork}$

Question: Is “she eats a fish with a fork” in $L(G)$?

Note: In above grammar, variables S is sentence, V is verb, N is noun, P is preposition, D is determiner, VP is verb phrase, NP is noun phrase, and PP is prepositional phrase.

Solution:

Construct a triangular table of the length of your given string/sentence.

X_{17}						
X_{16}	X_{27}					
X_{15}	X_{26}	X_{37}				
X_{14}	X_{25}	X_{36}	X_{47}			
X_{13}	X_{24}	X_{35}	X_{46}	X_{57}		
X_{12}	X_{23}	X_{34}	X_{45}	X_{56}	X_{67}	
X_{11}	X_{22}	X_{33}	X_{44}	X_{55}	X_{66}	X_{77}
She	eats	a	fish	with	a	fork

Filling 1st row: (i.e., cells representing sentence of length 1)

$X_{i,i}$ is a set of variables A such that $A \rightarrow w_i$ is a production of grammar G, where w_i is part of the given string ‘w’ or the whole string.

$X_{1,1} = NP$

$X_{2,2} = V, VP$

$X_{3,3} = D$

$X_{4,4} = N$

$X_{5,5} = P$

$X_{6,6} = D$

$X_{7,7} = N$

X_{17}						
X_{16}	X_{27}					
X_{15}	X_{26}	X_{37}				
X_{14}	X_{25}	X_{36}	X_{47}			
X_{13}	X_{24}	X_{35}	X_{46}	X_{57}		
X_{12}	X_{23}	X_{34}	X_{45}	X_{56}	X_{67}	
X_{11} N_p	X_{22} V, V_p	X_{33} D	X_{44} N	X_{55} P	X_{66} D	X_{77} N
<i>She</i>	<i>eats</i>	<i>a</i>	<i>fish</i>	<i>with</i>	<i>a</i>	<i>fork</i>

Filling 2nd row: (i.e., cells representing sentence of length 2)

$$X_{1,2} = (X_{i,i} \cdot X_{i+1,j}) = (X_{1,1} \cdot X_{2,2}) = \{N_p\} \cdot \{V, V_p\} = \{N_p V, N_p V_p\}$$

Look for production rules to generate $N_p V$ or $N_p V_p$.

$$X_{1,2} = \{ S \}$$

$$X_{2,3} = (X_{i,i} \cdot X_{i+1,j}) = (X_{2,2} \cdot X_{3,3}) = \{V, V_p\} \cdot \{D\} = \{ VD, V_p D \} = \emptyset$$

$$X_{3,4} = (X_{i,i} \cdot X_{i+1,j}) = (X_{3,3} \cdot X_{4,4}) = \{D\} \cdot \{N\} = N_p$$

$$X_{4,5} = (X_{i,i} \cdot X_{i+1,j}) = (X_{4,4} \cdot X_{5,5}) = \{N\} \cdot \{P\} = \emptyset$$

$$X_{5,6} = (X_{i,i} \cdot X_{i+1,j}) = (X_{5,5} \cdot X_{6,6}) = \{P\} \cdot \{D\} = \emptyset$$

$$X_{6,7} = (X_{i,i} \cdot X_{i+1,j}) = (X_{6,6} \cdot X_{7,7}) = \{D\} \cdot \{N\} = N_p$$

X_{17}						
X_{16}	X_{27}					
X_{15}	X_{26}	X_{37}				
X_{14}	X_{25}	X_{36}	X_{47}			
X_{13}	X_{24}	X_{35}	X_{46}	X_{57}		
X_{12} S	X_{23} Φ	X_{34} N_p	X_{45} Φ	X_{56} Φ	X_{67} N_p	
X_{11} N_p	X_{22} V, V_p	X_{33} D	X_{44} N	X_{55} P	X_{66} D	X_{77} N
<i>She</i>	<i>eats</i>	<i>a</i>	<i>fish</i>	<i>with</i>	<i>a</i>	<i>fork</i>

Filling 3rd row: (i.e., cells representing sentence of length 3)

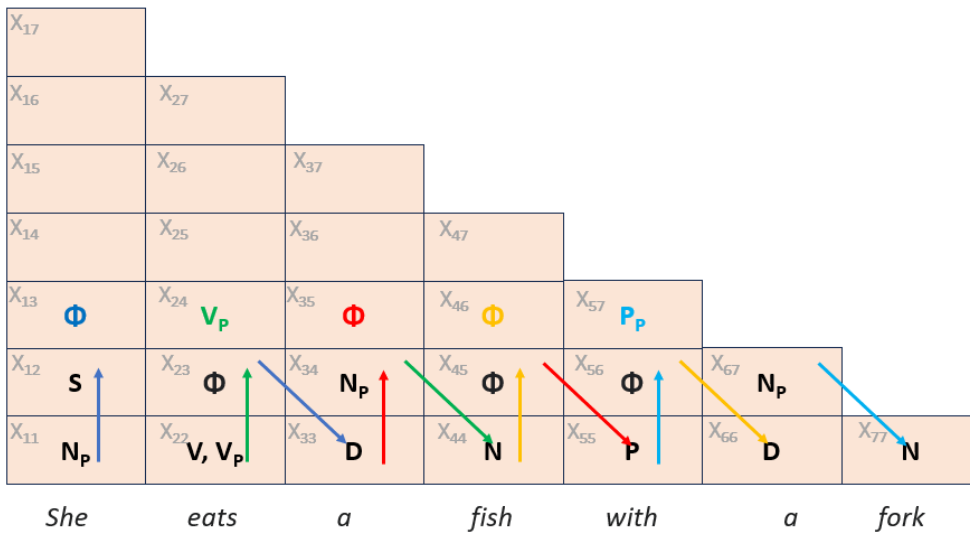
$$\begin{aligned}
X_{1,3} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
&= (X_{1,1} \cdot X_{2,3}) \cup (X_{1,2} \cdot X_{3,3}) \\
&= \{N_p\} \cdot \{\emptyset\} \cup \{S\} \cdot \{D\} \\
&= \emptyset \cup \emptyset \\
&= \emptyset
\end{aligned}$$

$$\begin{aligned}
X_{2,4} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
&= (X_{2,2} \cdot X_{3,4}) \cup (X_{2,3} \cdot X_{4,4}) \\
&= \{V, V_p\} \cdot \{N_p\} \cup \{\emptyset\} \cdot \{N\} \\
&= \{VN_p, V_p N_p\} \cup \emptyset \\
&= \{V_p\} \cup \emptyset \\
&= V_p
\end{aligned}$$

$$\begin{aligned}
X_{3,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
&= (X_{3,3} \cdot X_{4,5}) \cup (X_{3,4} \cdot X_{5,5}) \\
&= \{D\} \cdot \Phi \cup \{N_P\} \cdot \{P\} \\
&= \Phi \cup \{N_P P\} \\
&= \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{4,6} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
&= (X_{4,4} \cdot X_{5,6}) \cup (X_{4,5} \cdot X_{6,6}) \\
&= \{N\} \cdot \Phi \cup \Phi \cdot \{D\} \\
&= \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{5,7} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \\
&= (X_{5,5} \cdot X_{6,7}) \cup (X_{5,6} \cdot X_{7,7}) \\
&= \{P\} \cdot \{N_P\} \cup \Phi \cdot \{N\} \\
&= \{PN_P\} \cup \Phi \\
&= P_P \cup \Phi \\
&= P_P
\end{aligned}$$



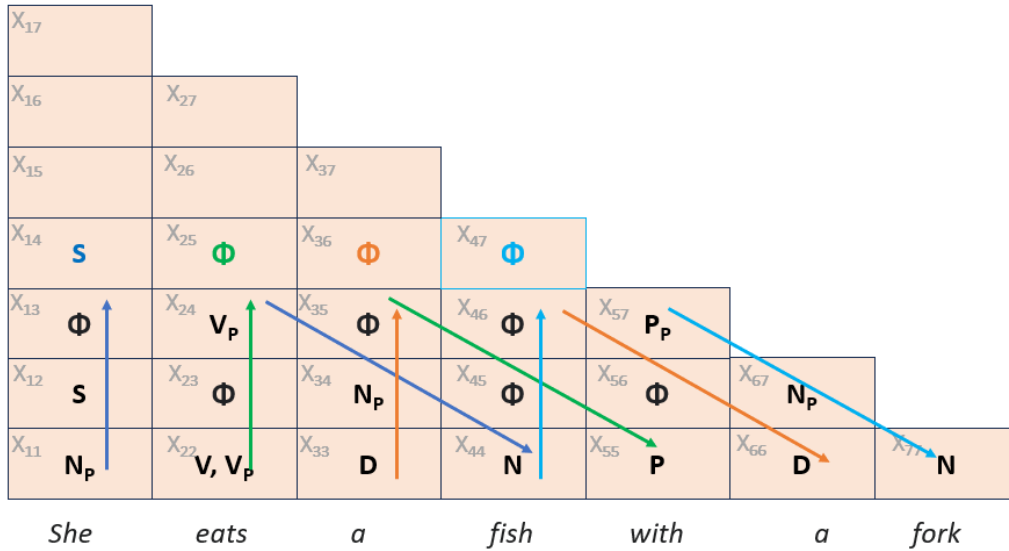
Filling 4th row: (i.e., cells representing sentence of length 4)

$$\begin{aligned}
X_{1,4} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \\
&= (X_{1,1} \cdot X_{2,4}) \cup (X_{1,2} \cdot X_{3,4}) \cup (X_{1,3} \cdot X_{4,4}) \\
&= \{N_P\} \cdot \{V_P\} \cup \{S\} \cdot \{N_P\} \cup \Phi \cdot \{N\} \\
&= \{N_P V_P\} \cup \{S N_P\} \cup \Phi \\
&= S \cup \Phi \cup \Phi \\
&= S
\end{aligned}$$

$$\begin{aligned}
X_{2,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \\
&= (X_{2,2} \cdot X_{3,5}) \cup (X_{2,3} \cdot X_{4,5}) \cup (X_{2,4} \cdot X_{5,5}) \\
&= \{V, V_P\} \cdot \Phi \cup \Phi \cdot \Phi \cup \{V_P\} \cdot \{P\} \\
&= \Phi \cup \Phi \cup \{V_P P\} \\
&= \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{3,6} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \\
&= (X_{3,3} \cdot X_{4,6}) \cup (X_{3,4} \cdot X_{5,6}) \cup (X_{3,5} \cdot X_{6,6}) \\
&= \{D\} \cdot \Phi \cup \{N_P\} \cdot \Phi \cup \Phi \cdot \{D\} \\
&= \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{4,7} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \\
&= (X_{4,4} \cdot X_{5,7}) \cup (X_{4,5} \cdot X_{6,7}) \cup (X_{4,6} \cdot X_{7,7}) \\
&= \{N\} \cdot \{P_P\} \cup \Phi \cdot \{N_P\} \cup \Phi \cdot \{N\} \\
&= \{NP_P\} \cup \Phi \cup \Phi \\
&= \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$



Filling 5th row: (i.e., cells representing sentence of length 5)

$$\begin{aligned}
X_{1,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \\
&= (X_{1,1} \cdot X_{2,5}) \cup (X_{1,2} \cdot X_{3,5}) \cup (X_{1,3} \cdot X_{4,5}) \cup (X_{1,4} \cdot X_{5,5}) \\
&= \{N_P\} \cdot \Phi \cup \{S\} \cdot \Phi \cup \Phi \cdot \Phi \cup \{S\} \cdot \{P\} \\
&= \Phi \cup \Phi \cup \Phi \cup \{SP\} \\
&= \Phi \cup \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{2,6} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \\
&= (X_{2,2} \cdot X_{3,6}) \cup (X_{2,3} \cdot X_{4,6}) \cup (X_{2,4} \cdot X_{5,6}) \cup (X_{2,5} \cdot X_{6,6}) \\
&= \{V, V_P\} \cdot \Phi \cup \Phi \cdot \Phi \cup \{V_P\} \cdot \Phi \cup \Phi \cdot \{D\} \\
&= \Phi \cup \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{1,5} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \\
&= (X_{1,1} \cdot X_{2,5}) \cup (X_{1,2} \cdot X_{3,5}) \cup (X_{1,3} \cdot X_{4,5}) \cup (X_{1,4} \cdot X_{5,5}) \\
&= \{D\} \cdot \Phi \cup \{N_P\} \cdot \{P_P\} \cup \Phi \cdot \{N_P\} \cup \Phi \cdot \{N\} \\
&= \Phi \cup \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

X ₁₇						
X ₁₆	X ₂₇					
X ₁₅ Φ	X ₂₆ Φ	X ₃₇ Φ				
X ₁₄ S	X ₂₅ Φ	X ₃₆ Φ	X ₄₇ Φ			
X ₁₃ Φ	X ₂₄ V _P	X ₃₅ Φ	X ₄₆ Φ	X ₅₇ P _P		
X ₁₂ S	X ₂₃ Φ	X ₃₄ N _P	X ₄₅ Φ	X ₅₆ Φ	X ₆₇ N _P	
X ₁₁ N _P	X ₂₂ V, V _P	X ₃₃ D	X ₄₄ N	X ₅₅ P	X ₆₆ D	X ₇₇ N
She	eats	a	fish	with	a	fork

Filling 6th row: (i.e., cells representing sentence of length 6)

$$\begin{aligned}
X_{1,6} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \cup (X_{i,i+4} \cdot X_{i+5,j}) \\
&= (X_{1,1} \cdot X_{2,6}) \cup (X_{1,2} \cdot X_{3,6}) \cup (X_{1,3} \cdot X_{4,6}) \cup (X_{1,4} \cdot X_{5,6}) \cup (X_{1,5} \cdot X_{6,6}) \\
&= \{N_P\} \cdot \Phi \cup \{S\} \cdot \Phi \cup \Phi \cdot \Phi \cup \{S\} \cdot \Phi \cup \Phi \cdot \{D\} \\
&= \Phi \cup \Phi \cup \Phi \cup \Phi \cup \Phi \\
&= \Phi
\end{aligned}$$

$$\begin{aligned}
X_{2,7} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \cup (X_{i,i+4} \cdot X_{i+5,j}) \\
&= (X_{2,2} \cdot X_{3,7}) \cup (X_{2,3} \cdot X_{4,7}) \cup (X_{2,4} \cdot X_{5,7}) \cup (X_{2,5} \cdot X_{6,7}) \cup (X_{2,6} \cdot X_{7,7}) \\
&= \{V, V_P\} \cdot \Phi \cup \Phi \cdot \Phi \cup \{V_P\} \cdot \{P_P\} \cup \Phi \cdot \{N_P\} \cup \Phi \cdot \{P\} \\
&= \Phi \cup \Phi \cup \{V_P P_P\} \cup \Phi \cup \Phi \\
&= \Phi \cup \Phi \cup \{V_P\} \cup \Phi \cup \Phi \\
&= V_P
\end{aligned}$$

X ₁₇						
X ₁₆ Φ	X ₂₇ V _P					
X ₁₅ Φ	X ₂₆ Φ	X ₃₇ Φ				
X ₁₄ S	X ₂₅ Φ	X ₃₆ Φ	X ₄₇ Φ			
X ₁₃ Φ	X ₂₄ V _P	X ₃₅ Φ	X ₄₆ Φ	X ₅₇ P _P		
X ₁₂ S	X ₂₃ Φ	X ₃₄ N _P	X ₄₅ Φ	X ₅₆ Φ	X ₆₇ N _P	
X ₁₁ N _P	X ₂₂ V, V _P	X ₃₃ D	X ₄₄ N	X ₅₅ P	X ₆₆ D	X ₇₇ N
She	eats	a	fish	with	a	fork

Filling 7th row: (i.e., cells representing sentence of length 7)

$$\begin{aligned}
X_{1,7} &= (X_{i,i} \cdot X_{i+1,j}) \cup (X_{i,i+1} \cdot X_{i+2,j}) \cup (X_{i,i+2} \cdot X_{i+3,j}) \cup (X_{i,i+3} \cdot X_{i+4,j}) \cup (X_{i,i+4} \cdot X_{i+5,j}) \\
&\quad \cup (X_{i,i+5} \cdot X_{i+6,j}) \\
&= (X_{1,1} \cdot X_{2,7}) \cup (X_{1,2} \cdot X_{3,7}) \cup (X_{1,3} \cdot X_{4,7}) \cup (X_{1,4} \cdot X_{5,7}) \cup (X_{1,5} \cdot X_{6,7}) \cup (X_{1,6} \cdot X_{7,7}) \\
&= \{N_P\} \cdot \{V_P\} \cup \{S\} \cdot \Phi \cup \Phi \cdot \Phi \cup \{S\} \cdot \{P_P\} \cup \Phi \cdot \{N_P\} \cup \Phi \cdot \{N\} \\
&= \{N_P V_P\} \cup \Phi \cup \Phi \cup \Phi \cup \Phi \cup \Phi \\
&= \{S\} \cup \Phi \cup \Phi \cup \Phi \cup \Phi \cup \Phi \\
&= S
\end{aligned}$$

X_{17} S						
X_{16} Φ	X_{27} V_P					
X_{15} Φ	X_{26} Φ	X_{37} Φ				
X_{14} S	X_{25} Φ	X_{36} Φ	X_{47} Φ			
X_{13} Φ	X_{24} V_P	X_{35} Φ	X_{46} Φ	X_{57} P_P		
X_{12} S	X_{23} Φ	X_{34} N_P	X_{45} Φ	X_{56} Φ	X_{67} N_P	
X_{11} N_P	X_{22} V, V_P	X_{33} D	X_{44} N	X_{55} P	X_{66} D	X_{77} N
She	eats	a	fish	with	a	fork

Is “she eats a fish with a fork” in $L(G)$?

Yes

We can see the table

the cell $X_{17} = S$ then

if $S \in X_{17}$ then “she eats a fish with a fork” $\in L(G)$

Exercises:

1) Apply CYK algorithm to determine whether the string 00111 is the member of the language generated by the grammar

$S \rightarrow XY$

$X \rightarrow YY \mid 0$

$Y \rightarrow XY \mid 1$

2) Apply CYK algorithm to determine whether the string 0011 is the member of the language generated by the grammar

$S \rightarrow XY$

$X \rightarrow YY \mid 0$

$Y \rightarrow XY \mid 1$

3) Apply CYK algorithm to determine whether the string 11000 is the member of the language generated by the grammar

$S \rightarrow PQ \mid QR$

$P \rightarrow QP \mid 0$

$Q \rightarrow RR \mid 1$

$R \rightarrow PQ \mid 0$

4) Apply CYK algorithm to determine whether the string 00000 is the member of the language generated by the grammar

$S \rightarrow PQ \mid QR$

$P \rightarrow QP \mid 0$

$Q \rightarrow RR \mid 1$

$R \rightarrow PQ \mid 0$

5) Apply CYK algorithm to determine whether the string aabbcc is the member of the language generated by the grammar.

$S \rightarrow AB$
 $A \rightarrow CD \mid CF$
 $B \rightarrow c \mid EB$
 $C \rightarrow a$
 $D \rightarrow b$
 $E \rightarrow c$
 $F \rightarrow AD$

6) Apply CYK algorithm to verify the given string (w) can be derived from the below grammar

$S \rightarrow AB \mid BC$
 $A \rightarrow BA \mid a$
 $B \rightarrow CC \mid b$
 $C \rightarrow AB \mid a$
w = baaba

References

- J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Second Edition, Addison Wesley, 2001
- T.A. Sudkamp, *An Introduction to the Theory of Computer Science Languages and Machines*, Third Edition, Addison Wesley, 2006
- <https://www.educative.io/answers/what-is-the-cyk-algorithm>
- <https://www.gatevidyalay.com/cyk-cyk-algorithm/>
- https://en.wikipedia.org/wiki/CYK_algorithm



Department of Computer Science and Engineering

PES University, Bangalore, India

UE23CS243A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Greibach Normal Form (GNF)

- A CFG is in Greibach normal form (GNF) if the right-hand sides of **all production rules start with a terminal symbol, optionally followed by some variables**.
- A non-strict form allows one exception to this format restriction for allowing the empty word (epsilon, ϵ) to be a member of the described language.
- The normal form was established by Sheila Greibach and it bears her name.
- A context-free grammar is in Greibach normal form, if all production rules are of the form:

$$A \rightarrow aA_1A_2...A_n$$

where A is a nonterminal symbol, a is a terminal symbol, $A_1A_2...A_n$ is a (possibly empty) sequence of nonterminal symbols.

- Every CFG can be transformed into an equivalent grammar in GNF. Various constructions exist.
- Given a grammar in GNF and **a derivable string with length n, any top-down parser will halt at depth n.**

GNF Definition (Non-strict form):

A CFG is in GNF if and only if, all of its production rules follow below types:

- $S \rightarrow \epsilon$, where S is a start symbol of the grammar, and ϵ means an empty string;
- $A \rightarrow a$, where A is a variable and a is a terminal symbol; and
- $A \rightarrow aA_1A_2...A_n$, where a is a terminal symbol and $A_1 A_2 \dots A_n$ are variables.

GNF Definition (Strict form):

A CFG is in GNF if and only all of its production rules follow below types:

- $A \rightarrow a$, where A is a variable and a is a terminal symbol; and
- $A \rightarrow aA_1A_2...A_n$, where a is a terminal symbol and $A_1 A_2 \dots A_n$ are variables.

Is G1 and G2 in GNF?

G1: $S \rightarrow xXY \mid xY$
 $X \rightarrow xX \mid x$

$$Y \rightarrow yY \mid y$$

$$\begin{aligned} \text{G2: } S &\rightarrow xXY \mid xY \\ X &\rightarrow xX \mid \epsilon \\ Y &\rightarrow yY \mid \epsilon \end{aligned}$$

The production rules of the G1 grammar satisfy the rules that are specified for GNF; thus, the **G1 grammar is in GNF**.

But the production rule of G2 grammar does not satisfy the rules that are specified for GNF as $X \rightarrow \epsilon$ and $Y \rightarrow \epsilon$ contain ϵ (only start symbol of the grammar can generate empty string (ϵ) in non-strict form). Thus, the grammar **G2 is not present in GNF**.

CFG into GNF Conversion Steps

Step 1: Eliminate lambda productions

Step 2: Eliminate Unit productions of type Variable $\rightarrow$ Variable

Step 3: Eliminate Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

Example 1: Convert the following CFG into GNF (Strict form)

$$S \rightarrow aSa \mid bSb \mid \lambda$$

Solution:

Step 1: Eliminate lambda productions

$$S \rightarrow aSa \mid bSb \mid \lambda \quad \Rightarrow \quad S \rightarrow aSa \mid bSb \mid aa \mid bb$$

Step 2: No Unit productions

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

Move Terminals to Unit Productions of type Variable $\rightarrow$ terminal

That is, for every terminal except the first symbol on the RHS of a non-unit production, add a substitute variable.

$$\begin{array}{lcl} S \rightarrow aSa \mid bSb \mid aa \mid bb & \Rightarrow & \begin{array}{l} S \rightarrow aSA \mid bSB \mid aA \mid bB \\ A \rightarrow a \\ B \rightarrow b \end{array} \end{array}$$

Example 2: Convert the following CFG into GNF (Non-Strict form)

$$S \rightarrow aSa \mid bSb \mid \lambda$$

Solution:

Step 1: Retain lambda production, because it's associated with start symbol

$$S \rightarrow aSa \mid bSb \mid \lambda$$

Step 2: No Unit productions

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

$$\begin{array}{l} S \rightarrow aSa \mid bSb \mid \lambda \\ A \rightarrow a \\ B \rightarrow b \end{array} \Rightarrow \begin{array}{l} S \rightarrow aSA \mid bSB \mid \lambda \\ A \rightarrow a \\ B \rightarrow b \end{array}$$

Example 3: Convert the following CFG into GNF

$$\begin{array}{l} S \rightarrow XY \mid Xn \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow Wn \mid o \end{array}$$

Solution:

Step 1: No lambda productions

Step 2: No Unit productions

Step 3: Eliminate Useless productions and Symbols

Eliminate useless symbol W (i.e., entire production where W is present), because W has no production and it's not generating anything.

$$\begin{array}{l} S \rightarrow XY \mid Xn \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow Wn \mid o \end{array} \Rightarrow \begin{array}{l} S \rightarrow XY \mid Xn \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow o \end{array}$$

Step 4: Convert the CFG to GNF as per definition.

- Replace, every nonterminal that appears first in the RHS of a production by its corresponding forms.

The production rules $S \rightarrow XY$ and $S \rightarrow Xn$ are not in GNF. Thus, substitute $X \rightarrow mX \mid m$

$$\begin{array}{l} S \rightarrow XY \mid Xn \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow o \end{array} \Rightarrow \begin{array}{l} S \rightarrow mXY \mid mY \mid mXn \mid mn \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow o \end{array}$$

- Move Terminals to Unit Productions of type Variable $\rightarrow$ terminal

That is, for every terminal except the first symbol on the RHS of a non-unit production, add a substitute variable.

The production rules $S \rightarrow mXn$ and $S \rightarrow mn$ are not in GNF. Thus, introduce new production $N \rightarrow n$, and replace n by N in production body.

$$\begin{array}{l} S \rightarrow mXY \mid mY \mid mXn \mid mn \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow o \end{array} \Rightarrow \begin{array}{l} S \rightarrow mXY \mid mY \mid mXN \mid mN \mid p \\ X \rightarrow mX \mid m \\ Y \rightarrow o \\ N \rightarrow n \end{array}$$

Example 4: Convert the following CFG into GNF

$$\begin{array}{l} S \rightarrow aA \mid bB \\ B \rightarrow bB \mid \lambda \\ A \rightarrow aA \mid \lambda \end{array}$$

Step 1: Eliminate lambda productions

$$S \rightarrow aA \mid bB \mid a \mid b$$

$$B \rightarrow bB \mid b$$

$$A \rightarrow aA \mid a$$

Step 2: No Unit productions

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

The grammar is already in GNF.

Example 5: Convert the following CFG into GNF

$$S \rightarrow AY \mid XX$$

$$X \rightarrow x \mid SX$$

$$Y \rightarrow y$$

$$A \rightarrow x$$

Solution:

Step 1: No lambda productions

Step 2: No Unit productions

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

The production rule $X \rightarrow SX$ is not in GNF. Thus, substitute $S \rightarrow AY \mid XX$ in the production rule $X \rightarrow SX$ as:

$S \rightarrow AY \mid XX$		$S \rightarrow AY \mid XX$
$X \rightarrow x \mid SX$	$\Rightarrow$	$X \rightarrow x \mid AYX \mid XXX$
$Y \rightarrow y$		$Y \rightarrow y$
$A \rightarrow x$		$A \rightarrow x$

The production rule $S \rightarrow AY$ and $Y \rightarrow AYX$ is not in GNF. Thus, substitute $A \rightarrow x$ in the production rule $S \rightarrow AY$ and $Y \rightarrow AYX$ as:

$S \rightarrow AY \mid XX$		$S \rightarrow xY \mid XX$
$X \rightarrow x \mid AYX \mid XXX$	$\Rightarrow$	$X \rightarrow x \mid xYX \mid XXX$ (The rule $X \rightarrow XXX$ has left recursion)
$Y \rightarrow y$		$Y \rightarrow y$
$A \rightarrow x$		$A \rightarrow x$ (This rule became useless, eliminate it)

We will now remove left recursion ($X \rightarrow XXX$) from the grammar.

[Left-recursion elimination formula:

$$A \rightarrow A\alpha \mid \beta_1 \mid \beta_2 \Rightarrow A \rightarrow \beta_1 B \mid \beta_2 B$$

$$B \rightarrow \alpha B \mid \epsilon$$

Where B is a newly introduced variable during recursion elimination]

$S \rightarrow xY \mid XX$		$S \rightarrow xY \mid XX$
$X \rightarrow XXX \mid x \mid xYX$	$\Rightarrow$	$X \rightarrow xZ \mid xYXZ$ (where Z is a newly introduced variable)
$Y \rightarrow y$		$Z \rightarrow XXZ \mid \epsilon$
		$Y \rightarrow y$

Removal of the null production $Z \rightarrow \epsilon$ would get:

$S \rightarrow xY \mid XX$		$S \rightarrow xY \mid XX$
$X \rightarrow xZ \mid xYXZ$	$\Rightarrow$	$X \rightarrow xZ \mid xYXZ \mid x \mid xYX$
$Z \rightarrow XXZ \mid \epsilon$		$Z \rightarrow XXZ \mid XX$
$Y \rightarrow y$		$Y \rightarrow y$

The production bodies XX and XXZ are not in GNF; thus, substitute $X \rightarrow xZ \mid xYXZ \mid x \mid xYX$ in production bodies XX and XXZ as:

$$\begin{array}{lcl} S \rightarrow xY \mid \textcolor{red}{XX} & & S \rightarrow xY \mid \textcolor{blue}{xZ}X \mid \textcolor{blue}{xYXZ}X \mid \textcolor{blue}{xX} \mid \textcolor{blue}{xYXX} \\ X \rightarrow \textcolor{blue}{xZ} \mid \textcolor{blue}{xYXZ} \mid \textcolor{blue}{x} \mid \textcolor{blue}{xYX} & \Rightarrow & X \rightarrow xZ \mid xYXZ \mid x \mid xYX \\ Z \rightarrow \textcolor{red}{XXZ} \mid \textcolor{red}{XX} & & Z \rightarrow \textcolor{blue}{xZ}XZ \mid \textcolor{blue}{xYXZ}XZ \mid \textcolor{blue}{xXZ} \mid \textcolor{blue}{xYXXZ} \mid \textcolor{blue}{xZ}X \mid \textcolor{blue}{xYXZ}X \mid \textcolor{blue}{xX} \mid \textcolor{blue}{xYXX} \\ Y \rightarrow y & & Y \rightarrow y \end{array}$$

Hence, this is the GNF form for the grammar G.

Note:

- Avoid ambiguous and/or left-recursive grammar for GNF conversion, even though you convert grammars with left-recursion and/or ambiguity to GNF, there is no use, the top-down parsers are not going to work.
- The below grammar is both ambiguous and left-recursive**

Example-1: Convert the following CFG into GNF (Strict form)

$$S \rightarrow aSa \mid bSb \mid SS \mid \lambda$$

Solution:

Step 1: Eliminate lambda productions

$$S \rightarrow aSa \mid bSb \mid SS \mid \lambda \quad \Rightarrow \quad S \rightarrow aSa \mid bSb \mid SS \mid S \mid aa \mid bb$$

Step 2: Eliminate Unit productions

$$S \rightarrow aSa \mid bSb \mid SS \mid aa \mid bb$$

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

$$\begin{array}{lcl} S \rightarrow aSa \mid bSb \mid \textcolor{red}{SS} \mid aa \mid bb & \Rightarrow & S \rightarrow aSa \mid bSb \mid \textcolor{red}{aSa}S \mid \textcolor{red}{bSb}S \mid aaS \mid bbS \mid aa \mid bb \\ S \rightarrow aSa \mid bSb \mid aSaS \mid bSbS \mid aaS \mid bbS \mid aa \mid bb & \Rightarrow & S \rightarrow aSA \mid bSB \mid aSAS \mid bSBS \mid aAS \mid bBS \mid aA \mid bB \\ & & A \rightarrow a \\ & & B \rightarrow b \end{array}$$

Example-2: Convert the following CFG into GNF (Non-Strict form)

$$S \rightarrow aSa \mid bSb \mid SS \mid \lambda$$

Solution:

Step 1: retain lambda production, because it's associated with start symbol of the grammar.

$$S \rightarrow aSa \mid bSb \mid SS \mid \lambda$$

Step 2: No Unit productions

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

$S \rightarrow aSa \mid bSb \mid SS \mid \lambda$



$S \rightarrow aSa \mid bSb \mid aSaS \mid bSbS \mid \lambda$

$S \rightarrow aSa \mid bSb \mid aSaS \mid bSbS \mid \lambda$



$S \rightarrow aSA \mid bSB \mid aSAS \mid bSBS \mid \lambda$

$A \rightarrow a$

$B \rightarrow b$

Exercises:

1) Give the relationship between the length of a string and the length of its derivation if the grammar is given in GNF?

2) Convert the following grammar into Greibach normal form (GNF).

$S \rightarrow XY1 \mid \emptyset$

$X \rightarrow \emptyset\emptyset X \mid Y$

$Y \rightarrow 1X1 \mid \lambda$

3) Convert the following grammar G into Greibach Normal Form (GNF).

$S \rightarrow CA \mid BB$

$B \rightarrow b \mid cB$

$C \rightarrow b$

$A \rightarrow a$

4) Consider the following CFG, find the number of productions in the grammar after it is converted into Greibach normal form(GNF).

$S \rightarrow XX \mid a$

$X \rightarrow aaS \mid b$

5) Convert the following grammar G into Greibach Normal Form (GNF).

$S \rightarrow XY1 \mid \emptyset$

$X \rightarrow \emptyset\emptyset X \mid 1$

$Y \rightarrow 1X1 \mid 1YY \mid XYY \mid \emptyset$

CFG (in GNF) to PDA conversion

Algorithm:

The process for converting a Context-Free Grammar (CFG) in GNF to a Push-Down Automata (PDA) involves the following steps:

- 1) Convert the CFG's productions into GNF
- 2) From the PDA start state (let's say q_0) without consuming any input symbol, push the start symbol on to the stack and make a transit to next state (let's say q_1).

$$\delta(q_0, \lambda, Z) = (q_1, SZ)$$

- 3) For every production of the form $A \rightarrow a\alpha$, add the transition $\delta(q_1, a, A) = (q_1, \alpha)$
- 4) Finally add the following transition $\delta(q_1, \lambda, Z) = (q_f, Z)$ where, q_f is the final state.

Note:

- The Pushdown Automata (PDA) constructed using above algorithm will have 3-states only.
- Stack contents are the variables in the sentential form of corresponding string derivation by the grammar.
- CFG and PDA are equivalent in power, a CFG generates a context-free language, and a PDA recognizes a context-free language.

Example 1: The following grammar should be converted to a PDA that supports the same language.

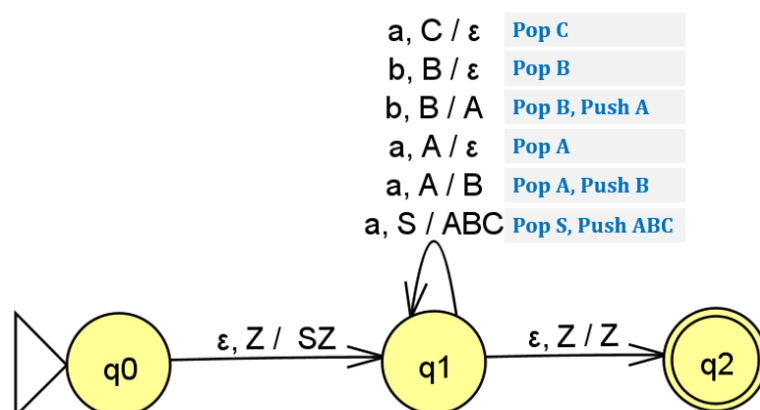
$S \rightarrow aABC$

$A \rightarrow aB \mid a$

$B \rightarrow bA \mid b$

$C \rightarrow a$

Solution: The given CFG is in GNF. Construct PDA



Check whether the string $w = 'aababa'$ is accepted by PDA or not.

First, we will check whether $w \in L(G)$ or not.

$S \Rightarrow aABC \Rightarrow aaBC \Rightarrow aabAC \Rightarrow aabaBC \Rightarrow aababC \Rightarrow aababa$

$\delta(q_0, aababa, Z) \vdash \delta(q_1, aababa, SZ)$

$\vdash \delta(q_1, ababa, ABCZ) \rightarrow \vdash \delta(q_1, baba, BCZ) \rightarrow \dots$ (Path dies or stuck)

$\vdash \delta(q_1, baba, BBCZ) \rightarrow \vdash \delta(q_1, aba, BCZ) \rightarrow \dots$ (Paths dies or stuck)

$\vdash \delta(q_1, aba, ABCZ) \rightarrow \vdash \delta(q_1, ba, BBCZ) \rightarrow \dots$ (Paths dies or stuck)

$\vdash \delta(q_1, ba, BCZ) \rightarrow \vdash \delta(q_1, a, ACZ) \rightarrow \dots$ (Paths dies or stuck)

$\vdash \delta(q_1, a, CZ)$

$\vdash \delta(q_1, \epsilon, Z)$

$\vdash \delta(q_2, \epsilon, Z)$

Accept

Example 2: The following grammar should be converted to a PDA that supports the same language.

$S \rightarrow 0S1 \mid 1S0 \mid \epsilon$ (Non-strict form)

Solution:

Step 1: No lambda productions with non-starting symbols.

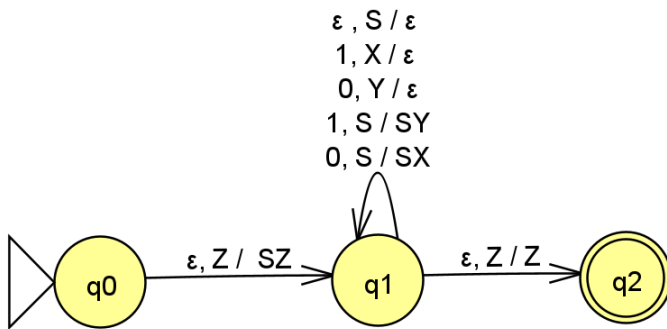
Step 2: No Unit productions

Step 3: No Useless productions and Symbols

Step 4: Convert the CFG to GNF as per definition.

$S \rightarrow 0S1 \mid 1S0 \mid \epsilon \quad \Rightarrow \quad \begin{array}{l} S \rightarrow 0SX \mid 1SY \mid \epsilon \\ X \rightarrow 1 \\ Y \rightarrow 0 \end{array}$

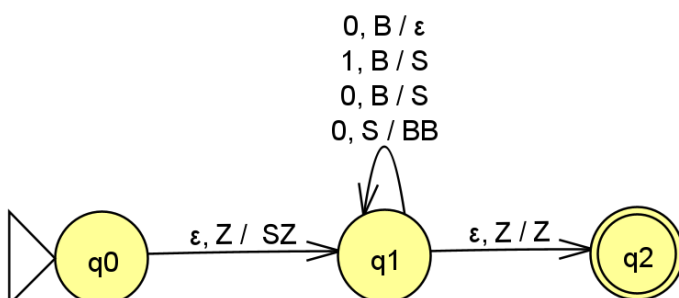
A PDA could be:



Example 3: Create a PDA for the provided CFG, and check if 010^4 is supported by it.

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$



$\delta(q_0, \epsilon, Z) = \{(q_1, SZ)\}$

$$\delta(q_1, 0, S) = \{(q_1, BB)\}$$

$$\delta(q_1, 0, B) = \{(q_1, \varepsilon) \mid (q_1, S)\}$$

$$\delta(q_1, 1, B) = \{(q_1, S)\}$$

$$\delta(q_1, \varepsilon, Z) = \{(q_2, Z)\}$$

Analyzing 010⁴, i.e., 010000, against a PDA

$$\begin{aligned} \delta(q_0, 010000, Z) &\vdash \delta(q_1, 010000, SZ) \\ &\vdash \delta(q_1, 10000, BBZ) \\ &\vdash \delta(q_1, 0000, SBZ) \\ &\vdash \delta(q_1, 000, BBBZ) \rightarrow \vdash \delta(q_1, 00, SBBZ) \rightarrow \dots \text{(Path dies or stuck)} \\ &\vdash \delta(q_1, 00, BBZ) \rightarrow \vdash \delta(q_1, 0, SBZ) \rightarrow \dots \text{(Path dies or stuck)} \\ &\vdash \delta(q_1, 0, BZ) \rightarrow \vdash \delta(q_1, \varepsilon, SZ) \rightarrow \dots \text{(Path dies or stuck)} \\ &\vdash \delta(q_1, \varepsilon, Z) \\ &\vdash \delta(q_2, \varepsilon, Z) \\ &\quad \text{Accept} \end{aligned}$$

Exercises:

1) Construct PDA for the following CFG and check if 000010101000100011 is supported by it.

$$S \rightarrow \mathbf{ooS1} \mid \mathbf{oSo1} \mid \mathbf{o1So} \mid \mathbf{oS1o} \mid \mathbf{1oS0} \mid \mathbf{1Soo} \mid \lambda$$

2) The following CFG should be converted to a PDA that supports the same language.

$$\begin{aligned} S &\rightarrow \mathbf{oAB1} \\ A &\rightarrow \mathbf{BAS} \mid \lambda \\ B &\rightarrow \mathbf{o} \mid \mathbf{1} \end{aligned}$$

3) The following CFG should be converted to a PDA that supports the same language.

$$\begin{aligned} S &\rightarrow \mathbf{oXYX} \mid \mathbf{oYY} \\ X &\rightarrow \mathbf{1X} \mid \mathbf{1} \\ Y &\rightarrow \mathbf{2Y} \mid \mathbf{2} \end{aligned}$$

4) The following CFG should be converted to a PDA that supports the same language.

$$S \rightarrow \mathbf{aSb} \mid \mathbf{a} \mid \mathbf{b} \mid \lambda$$



Department of Computer Science and Engineering

PES University, Bangalore, India

UE23CS242A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Closure Properties of Context-Free Languages

Context Free Languages (CFLs) are accepted by pushdown automata. Context free languages can be generated by context free grammars.

We will show that CFLs are closed under

1. Union
2. Concatenation
3. Kleene star
4. Reversal and
5. Homomorphism

But CFLs are not closed under

1. Intersection
2. Difference and
3. Complement

1. Union

In terms of languages:

If L_1 and L_2 are two context free languages, their union $L_1 \cup L_2$ will also be context free.

Example 1:

$L_1 = \{ a^n b^n c^m \mid m \geq 0 \text{ and } n \geq 0 \}$ and $L_2 = \{ a^n b^m c^m \mid n \geq 0 \text{ and } m \geq 0 \}$

$L_3 = L_1 \cup L_2 = \{ a^n b^n c^m \cup a^n b^m c^m \mid n \geq 0, m \geq 0 \}$ is also context free language.

L_1 says number of a's should be equal to number of b's and L_2 says number of b's should be equal to number of c's. Their union says either of two conditions to be true. So, it is also context free language.

So CFLs are closed under Union.

Example 2:

$L_1 = \{ a^n b^n \mid n \geq 0 \}$ and $L_2 = \{ c^m d^m \mid m \geq 0 \}$

$L_3 = L_1 \cup L_2 = \{ a^n b^n \cup c^m d^m \mid n \geq 0 \text{ and } m \geq 0 \}$ is also context free language.

L_1 says number of a's should be equal to number of b's and L_2 says number of c's should be equal to number of d's. Their union says either of two conditions to be true. So, it is also context free language.

So CFLs are closed under Union.

In terms of grammars:

Suppose $G_1 = (V_1, \Sigma_1, P_1, S_1)$ and $G_2 = (V_2, \Sigma_2, P_2, S_2)$.

Where P_1 is

$$S_1 \rightarrow aS_1b \mid \epsilon$$

Where P_2 is

$$S_2 \rightarrow cS_2d \mid \epsilon$$

Then $L(G_1) = \{a^n b^n : n \geq 0\}$. Also, $L(G_2) = \{c^m d^m : m \geq 0\}$.

Form a new grammar for $G_1 \cup G_2$ by combining all the symbols and productions of G_1 and G_2 . Then, add a new start symbol S .

$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P, S)$

Where P is

$$S \rightarrow S_1 \mid S_2$$

$$S_1 \rightarrow aS_1b \mid \epsilon$$

$$S_2 \rightarrow cS_2d \mid \epsilon$$

Then $L(G) = \{a^n b^n \cup c^m d^m \mid m, n \geq 0\}$ is also context free language.

2. Concatenation

In terms of languages:

If L_1 and L_2 are two context free languages, their concatenation $L_1.L_2$ will also be context free.

For example,

$L_1 = \{a^n b^n \mid n \geq 0\}$ and $L_2 = \{c^m d^m \mid m \geq 0\}$

$L_3 = L_1.L_2 = \{a^n b^n c^m d^m \mid m \geq 0 \text{ and } n \geq 0\}$ is also context free language.

L_1 says number of a's should be equal to number of b's and L_2 says number of c's should be equal to number of d's.

Their concatenation says first number of a's should be equal to number of b's, then number of c's should be equal to number of d's.

So, we can create a PDA which will first push for a's, pop for b's, push for c's then pop for d's. So it can be accepted by pushdown automata, hence context free.

So CFLs are closed under Concatenation.

In terms of grammars:

Suppose $G_1 = (V_1, \Sigma_1, P_1, S_1)$ and $G_2 = (V_2, \Sigma_2, P_2, S_2)$.

Where P_1 is

$$S_1 \rightarrow aS_1b \mid \epsilon$$

Where P_2 is

$$S_2 \rightarrow cS_2d \mid \epsilon$$

Then $L(G_1) = \{a^n b^n : n \geq 0\}$. Also, $L(G_2) = \{c^m d^m : m \geq 0\}$.

Form a new grammar for $G_1.G_2$ by combining all the symbols and productions of G_1 and G_2 . Then, add a new start symbol S .

$$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P, S)$$

Where P is

$$S \rightarrow S_1 S_2$$

$$S_1 \rightarrow a S_1 b \mid \epsilon$$

$$S_2 \rightarrow c S_2 d \mid \epsilon$$

Then $L(G) = \{a^n b^n c^m d^m \mid m, n \geq 0\}$ is also context free language.

3. Kleene star

In terms of languages:

If L is context free, its Kleene closure L^* will also be context free.

For example,

$$L = \{a^n b^n \mid n \geq 0\}$$

$$L = \{\epsilon, ab, aabb, aaabbb, aaaabbbb, \dots\}$$

$$L^* = (\epsilon + ab + aabb + aaabbb + \dots)^*$$

$$L^* = \{\epsilon, ab, abab, abaaabbb, abaabbab, \dots\}$$
 is also a CFL.

So CFL is closed under Kleen Closure.

Assignment: Construct PDA for L^*

In terms of grammars:

$$\text{Suppose } G = (V, \Sigma, P, A)$$

Where P is

$$A \rightarrow aAb \mid \epsilon$$

$$\text{Then } L(G) = \{a^n b^n : n \geq 0\}$$

Form a new grammar for G^* (that generates L^*). Then, add a new start symbol S .

$$G^* = (V \cup \{S\}, \Sigma, P', S)$$

Where P' is

$$S \rightarrow SA \mid \epsilon$$

$$A \rightarrow aAb \mid \epsilon$$

Then $L(G^*) = \{a^n b^n \mid n \geq 0\}^*$ is also context free language.

Assignment: Construct PDA for L^* (i.e., $L(G^*)$)

4. Reversal

In terms of languages:

If L_1 is context free, its reversal L_1^R will also be context free. For example,

$$L_1 = \{a^n b^n \mid n \geq 1\}$$

$$L_1^R = \{b^n a^n \mid n \geq 1\}$$
 is also context free language.

So CFL are closed under reversal.

In terms of grammars:

$$\text{Suppose } G_1 = (V_1, \Sigma_1, P_1, S_1)$$

Where P_1 is

$$S_1 \rightarrow aS_1b \mid ab$$

Then $L(G_1) = \{a^n b^n : n \geq 1\}$

Form a new grammar for G_1^R . If L_1 is a CFL with grammar G_1 , form a grammar for G_1^R by reversing the right side of every production.

$$G_1^R = (V_1, \Sigma_1, P, S_1)$$

Where P is

$$S_1 \rightarrow bS_1a \mid ba$$

Then $L(G_1^R) = \{b^n a^n \mid n \geq 1\}$ is also context free language.

5. Homomorphism

Homomorphism is a function that maps a string of symbols in a language to a string of symbols in another language.

A homomorphism is a function $h: \Sigma^* \rightarrow \Delta^*$ defined as follows:

- $h(\epsilon) = \epsilon$
- For $a \in \Sigma$, $h(a)$ is any string in Δ^*

Example: $h: \{0, 1\}^* \rightarrow \{a, b\}^*$ where $h(0) = aa$ and $h(1) = bb$

A homomorphism on an alphabet is a function that gives a string for each symbol in that alphabet.

- For $w = a_1 a_2 \dots a_n \in \Sigma^* (n \geq 2)$, $h(w) = h(a_1)h(a_2) \dots h(a_n)$

A homomorphism h maps a string $a \in \Sigma^*$ to a string in Δ^* by mapping each character of a to a string $h(a) \in \Delta^*$

A homomorphism is a function that maps strings to strings while respecting concatenation.

For any string $xyx \in \Sigma^*$, $h(xyx) = h(x).h(y).h(x)$

Example: $h: \{0, 1\}^* \rightarrow \{a, b\}^*$ where $h(0) = ab$ and $h(1) = ba$.

Then $h(0011) = h(0).h(0).h(1).h(1) = ababbaba$

Homomorphism as an Operation on Languages

Given a homomorphism $h: \Sigma^* \rightarrow \Delta^*$ and a language $L \subseteq \Sigma^*$, define $h(L) = \{h(w) \mid w \in L\} \subseteq \Delta^*$.

Example 1:

$L = \{0^n 1^n \mid n \geq 0\}$ is a CFL. where $\Sigma = \{0, 1\}$ and $\Delta = \{a, b\}$

Let, $h(0) = a$ and $h(1) = b$. Then $h(L) = \{a^n b^n \mid n \geq 0\}$ is also CFL.

Example 2:

$L = \{(a+b)^n \mid n \leq 2\} = \{\epsilon, a, b, aa, bb, ab, ba\}$ is a CFL, where $\Sigma = \{a, b\}$ and $\Delta = \{0, 1\}$

Let, $h(a) = 01$ and $h(b) = 10$. Then $h(L) = \{(01+10)^n \mid n \leq 2\} = \{\epsilon, 01, 10, 0101, 1010, 0110, 1001\}$ is also CFL.

Homomorphism as an Operation on Grammars

Given a homomorphism $h: \Sigma^* \rightarrow \Delta^*$, where $\Sigma = \{0, 1\}$ and $\Delta = \{a, b\}$

Example 1:

Let, CFG $G = (V, \Sigma, P, S)$ generating $L(G)$ has productions $S \rightarrow 0S1 \mid 01$

Let, $h(0) = ab$ and $h(1) = \epsilon$

$h(L(G))$ has the grammar $G' = (V', \Gamma, P', S)$ with productions

$$S \rightarrow abS \mid ab$$

Example 2:

Let, CFG $G = (V, \Sigma, P, S)$ generating $L(G)$ has productions $S \rightarrow 0S0 \mid 1S1 \mid \epsilon$

Let, $h(0) = aba$ and $h(1) = bb$

$h(L(G))$ has the grammar $G' = (V', \Gamma, P', S)$ with productions

$S \rightarrow X_0 S X_0 \mid X_1 S X_1 \mid \epsilon$

$X_0 \rightarrow \mathbf{aba}$

$X_1 \rightarrow \mathbf{bb}$

Note:

Inverse Homomorphism: Given homomorphism $h: \Sigma^* \rightarrow \Delta^*$ and $L \subseteq \Delta^*$, $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L\}$.

$h^{-1}(L)$ consists of strings whose homomorphic images are in L .

The inverse homomorphism maps every string in the language to a unique string in the other language.

Non-closure properties of CFL

Context-free languages are not closed under intersection or complement.

6. Intersection

If L_1 and L_2 are two context free languages, their intersection $L_1 \cap L_2$ need not be context free.

For example,

$L_1 = \{a^n b^n c^m \mid m, n \geq 0\}$ // equal a's and b's.

$L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$ // equal b's and c's

$L_3 = L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$ // need not be context free.

L_1 says number of a's should be equal to number of b's and L_2 says number of b's should be equal to number of c's. Their intersection says both conditions need to be true, but push down automata can compare only two. **So, it cannot be accepted by pushdown automata**, hence not context free.

Similarly, complementation of context free language L_1 which is $\Sigma^* - L_1$, need not be context free.

7. Complement

We can prove something more general:

Closure under complement implies closure under intersection, since:

$$L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$$

Thus, **if CFL's were closed under complement, they would be closed under intersection (which they are not).**

8. Difference

We can prove something more general:

Any class of languages that is closed under difference is closed under intersection.

Proof: $L_1 \cap L_2 = L_1 - (L_1 - L_2)$

Thus, **if CFL's were closed under difference, they would be closed under intersection, but they are not.**

Note 1:

- If CFLs were closed under complement, they would also be closed under intersection.
- Therefore, CFLs are not closed under complementation because they are not closed under intersection.

Note 2:

- **CFLs are closed under**

- union,
- concatenation,
- kleene star,
- reversal,
- homomorphism and
- Inverse homomorphism
- CFLs are not closed under
 - intersection,
 - difference and
 - complement.

CFL intersection with a regular language

The intersection of a context-free language and a regular language is context free. The idea of the proof is to simulate a push-down automaton and a finite state automaton in parallel and only accept if both machines accept.

Intersection of two CFL's need not be context free.

But the intersection of a CFL with a regular language is always a CFL.

Example 1:

$L_1 = \{ a^*b^* \}$ is a regular language

$L_2 = \{ a^n b^n, n \geq 0 \}$ is a context free language

$L_3 = L_1 \cap L_2 = \{ a^n b^n, n \geq 0 \}$ is a context free language

Example 2:

$L_1 = \{ a^n b^n, n \geq 0 \}$ is a context free language

$L_2 = \{ a^{100} b^{100} \}$ is a regular language

$L_2^c = \{ (a+b)^* \} - \{ a^{100} b^{100} \}$ is a regular language

$L_3 = L_1 \cap L_2^c = \{ a^n b^n \mid n \neq 100, n \geq 0 \}$ is a context free language

Assignment:

- Construct DFA for $L_2 = \{ a^{100} b^{100} \}$
- Construct PDA for $L_3 = \{ a^n b^n \mid n \neq 100, n \geq 0 \}$

Example 3:

$L_1 = \{ w \mid n_a(w) = n_b(w) \}$ is a context free language

$L_2 = \{ a^* b^* \}$ is a regular language

$L_3 = L_1 \cap L_2 = \{ a^n b^n, n \geq 0 \}$ is a context free language

Example 4:

$L_1 = \{ a^n b^n, n \geq 0 \}$ is a context free language

$L_2 = \{ w \in \{a, b\}^* : |w| \text{ is divisible by } 3 \}$ is a regular language

$L_3 = \{ a^n b^n \mid n \geq 0 \} \cap \{ w \in \{a, b\}^* : |w| \text{ is divisible by } 3 \}$

$L_3 = \{ a^{3n} b^{3n} \mid n \geq 0 \}$ is context-free language.

Assignment: Construct PDA for $L_3 = \{a^{3n}b^{3n} \mid n \geq 0\}$

Closure Properties of Deterministic Context-Free Languages (DCFLs)

- **DCFLs are closed only under**
 - **Complement** and
 - **Inverse homomorphism**
- **DCFLs are not closed under**
 - **union,**
 - **intersection,**
 - **concatenation,**
 - **kleene star,**
 - **reversal, and**
 - **homomorphism**

Deterministic Context-Free Languages (DCFLs)

Deterministic CFLs are subset of CFL which can be recognized by Deterministic PDA.

Deterministic PDA has only one move from a given state and input symbol, i.e., it does not have choice.

For a language to be DCFL it should be clear when to PUSH or POP.

Note: Choose DCFLs in such a way that after applying the operation, the DCFLs are not closed under that operation.

1. Union

For example,

$L_1 = \{a^n b^n c^m \mid m > 0 \text{ and } n > 0\}$ is a DCFL because for a's, we can push on stack and for b's we can pop. It can be recognized by Deterministic PDA.

$L_2 = \{a^n b^m c^m \mid m > 0 \text{ and } n > 0\}$ is a DCFL because for b's, we can push on stack and for c's we can pop. It can be recognized by Deterministic PDA.

Let, **$L_3 = L_1 \cup L_2 = \{a^n b^n c^m \cup a^n b^m c^m \mid n > 0, m > 0\}$ cannot be recognized by DPDA** because either number of a's and b's can be equal or either number of b's and c's can be equal and both the languages strings begins with the same symbol. So, it can only be implemented by NPDA. **Thus, it is CFL but not DCFL.**

So, DCFLs are not closed under union.

2. Concatenation

For example,

$L_1 = \{a^m b^n \mid m < n\}$ is a DCFL

$L_2 = \{wcw^R \mid w \in \{a, b\}^*\}$ is a DCFL

Let, $L_3 = L_1.L_2 = \{a^m b^n wcw^R \mid \text{where } m < n \text{ and } w \in \{a, b\}^*\}$

The DPDA of L_3 would not be knowing which part (L_1 or L_2 string, because both made up of same alphabet) it is currently processing and would need non-determinism to correctly recognize the strings of the newly formed language. **Therefore, L_3 is CFL but not a DCFL.**

So, DCFLs are not closed under concatenation.

3. Kleene star

For example,

$L_1 = \{ca^n b^n \mid n \geq 0\}$ is a DCFL

$L_2 = \{a^m b^{2m} \mid m \geq 1\}$ is a DCFL

$L = L_1 \cup L_2 = \{ca^n b^n \cup a^m b^{2m} \mid n \geq 0, m \geq 1\}$ is a DCFL

$L = \{c, cab, caabb, \dots, abb, aabbbb, \dots\}$ is a DCFL

$L^* = (c + cab + caabb + \dots + abb + aabbbb + \dots)^*$

$L^* = \{\epsilon, c, cc, \dots cab, cabcab, \dots ccab, cabb, ccabbabb, \dots abbabb, \dots\}$

In order to recognize the above words, you would have to form a new automaton, by connecting the individual automata appropriately. However, this connection cannot be made deterministically.

So DCFL is not closed under Kleen Closure.

4. Reversal

For example,

$L = \{wcw^r(a|b)^* \mid w \in \{a, b\}^+\}$ is a DCFL

$L^R = \{(a|b)^*w^rcw \mid w \in \{a, b\}^+\}$ is not a DCFL

You don't know where the string of $(a+b)^*$ ends, where the string of w^rcw (odd palindrome) begins because first part of $L^R \in \{a, b\}^*$ and second palindrome part of $L^R \in \{a, b\}^+$

So DCFL is not closed under reversal.

5. Intersection

For example,

$L_1 = \{a^m b^n c^m \mid m > 0 \text{ and } n > 0\}$ is DCFL

$L_2 = \{a^m b^n c^n \mid n > 0 \text{ and } m > 0\}$ is DCFL

$L_3 = L_1 \cap L_2 = \{a^n b^n c^n \mid n > 0\}$ is not a CFL only.

So DCFL is not closed under intersection.

6. Homomorphism

Given a homomorphism $h: \Sigma^* \rightarrow \Delta^*$ and a language $L \subseteq \Sigma^*$, define $h(L) = \{h(w) \mid w \in L\} \subseteq \Delta^*$.

Example 1:

$L = \{0^n 1^n \mid n > 0\}$ is a DCFL.

where $\Sigma = \{0, 1\}$ and $\Delta = \{a, b\}$

Let, $h(0) = aa$ and $h(1) = ab$.

Then $h(L) = \{(aa)^n (ab)^n \mid n > 0\}$ is not a DCFL.

So DCFL is not closed under homomorphism.

7. Complement

The DCFLs are closed under complement.

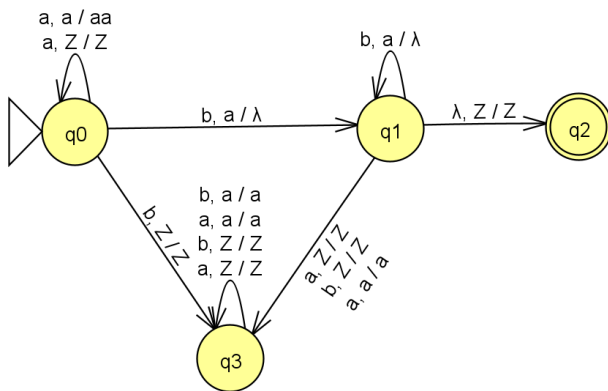
In order to take the complement, the DPDA of the corresponding DCFL must be complete (including trap state for invalid configurations) and then toggle final and non-final states.

Example 1:

$L = \{a^n b^n \mid n > 0\}$ is a DCFL.

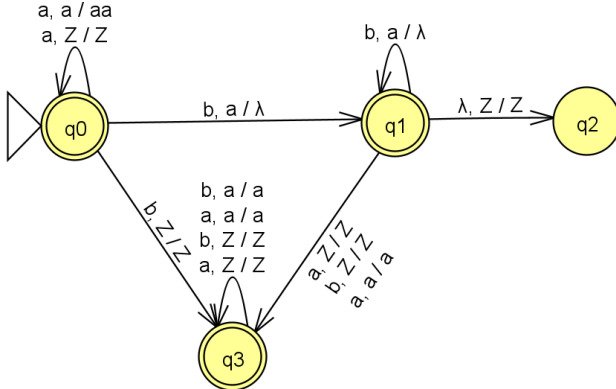
$L^c = \Sigma^* - L$

DPDA(L):



Invalid strings: b^4a^4 , a^3b^4 , a^4b^3 , $a^4b^4ab^3$, $a^4b^4a^3$...

DPDA(L^c):



The following properties of CFLs are undecidable

Note: Undecidability means there is no algorithm that provides correct answer for all inputs.

1. Given a CFG G , is it possible to determine G is ambiguous or not?

Solution: Not possible

There is no algorithm to predict whether a given grammar is ambiguous or not.

2. Given two CFLs L_1 and L_2 , is it possible to determine whether $L_1 = L_2$ i.e., whether they generate the same language?

Solution:

If not, we will eventually get an answer (i.e., not equal). If yes, we will never terminate as both the languages are infinite.

Note: There is no “algorithm” to determine if two CFGs generate the same language.

3. Given two infinite CFLs L_1 and L_2 , is it possible to determine if $L_1 \cap L_2 = \Phi$?

Solution:

if the languages have string in common, we will eventually get an answer, if not we will wait forever as the languages are infinite.

Decision Properties of CFLs:

- **Test for finiteness:** Decidable
- **Test for Emptiness:** Decidable
- **Test for Membership:** Decidable.

Is it possible to write an algorithm to answer the following about context free language(s)?

1. Given a CFL L, is it possible to determine if L is infinite?

Solution: If CFG G is recursive (directly or indirectly), then G generates infinite language.

Or If the machine (after eliminating dead and unreachable states) of L contains loops or cycles then the language L is infinite.

2. Given a CFL L, is it possible to determine if L is empty i.e. $L = \Phi$?

Solution: If start symbol of grammar G for L is found to be useless, then $L(G)$ is empty.

Or If machine of L is free from final state, then machine accepts empty language.

3. Given a CFG G and a string w, is it possible to determine if $w \in L(G)$?

Solution: Using CYK parsing

4. Given a CFL L, is it possible to determine if L generates everything, i.e. $L = \Sigma^*$?

Solution: Decidable for DCFLs but not for CFLs as DCFLs are closed under complement. (That is L^c should be Φ)

END

Question 1: Consider the language L1, L2 and L3 as given below.

- $L1 = \{ a^m b^n \mid m, n \geq 0 \}$
- $L2 = \{ a^n b^n \mid n \geq 0 \}$
- $L3 = \{ a^n b^n c^n \mid n \geq 0 \}$

Which of the following statements is NOT TRUE?

- A. Push Down Automata (PDA) can be used to recognize L1 and L2
- B. L1 is a regular language
- C. All the three languages are context free
- D. Turing machine can be used to recognize all the three languages

Solution :

Option (A) says PDA can be used to recognize L1 and L2.

L1 contains all strings with any no. of a followed by any no. of b. So, it can be accepted by PDA. L2 contains strings with n no. of a's followed by n no. of b's. It can also be accepted by PDA. So, option (A) is correct.

Option (B) says that L1 is regular.

It is true as regular expression for L1 is a^*b^* .

Option (C) says L1, L2 and L3 are context free.

L3 languages contains all strings with n no. of a's followed by n no. of b's followed by n no. of c's. But it can't be accepted by PDA. So, option (C) is not correct.

Option (D) is correct as Turing machine can be used to recognize all the three languages.

Question 2: The language $L = \{ 0^i 12^i \mid i \geq 0 \}$ over the alphabet $\{0, 1, 2\}$ is :

- A. Not recursive
- B. Is recursive and deterministic CFL
- C. Is regular
- D. Is CFL but not deterministic CFL.

Solution: The above language is deterministic CFL as for 0's, we can push 0 on stack and for 2's we can pop corresponding 0's. As there is no ambiguity which moves to take, it is deterministic.

So, correct option is (B). As CFL is subset of recursive, it is recursive as well.

Question 3: Consider the following languages:

$$L1 = \{ 0^n 1^n \mid n > 0 \}$$

$$L2 = \{ w c w^r \mid w \in \{a,b\}^* \}$$

$$L3 = \{ w w^r \mid w \in \{a,b\}^* \}$$

Which of these languages are deterministic context-free languages?

A. None of the languages

B. Only L1

C. Only L1 and L2

D. All three languages

Solution:

L1 contains all strings in which n 0's are followed by n 1's. Deterministic PDA can be constructed to accept L1. For 0's we can push it on stack and for 1's, we can pop from stack. Hence, it is DCFL.

L2 contains all strings of form $w c w^r$ where w is a string of a's and b's and w^r is reverse of w. For example, aabbcbbaa. To accept this language, we can construct PDA which will push all symbols on stack before c. After c, if symbol on input string matches with symbol on stack, it is popped. So, L2 can also be accepted with deterministic PDA, hence it is also DCFL.

L3 contains all strings of form $w w^r$ where w is a string of a's and b's and w^r is reverse of w. But we don't know where w ends and w^r starts. e.g.; aabbaa is a string corresponding to L3. For first a, we will push it on stack. Next a can be either part of w or w^r where $w=a$. So, there can be multiple moves from a state on an input symbol. So, only non-deterministic PDA can be used to accept this type of language. Hence, it is NCFL not DCFL.

So, correct option is (C). Only, L1 and L2 are DCFL.

Question 4: Which one of the following grammars generate the language $L = \{ a^i b^j \mid i \neq j \}$

$$S \rightarrow AC \mid CB, C \rightarrow aCb \mid a \mid b, A \rightarrow aA \mid \epsilon, B \rightarrow Bb \mid \epsilon$$

$$S \rightarrow aS \mid Sb \mid a \mid b$$

$$S \rightarrow AC \mid CB, C \rightarrow aCb \mid \epsilon, A \rightarrow aA \mid \epsilon, B \rightarrow Bb \mid \epsilon$$

$$S \rightarrow AC \mid CB, C \rightarrow aCb \mid \epsilon, A \rightarrow aA \mid a, B \rightarrow Bb \mid b$$

Solution: The best way to solve these type of questions is to eliminate options which do not satisfy conditions. The conditions for language L is no. of a's and no. of b's should be unequal.

In option (B), $S \Rightarrow aS \Rightarrow ab$. It can generate strings with equal a's and b's. So, this option is incorrect.

In option (C), $S \Rightarrow AC \Rightarrow C \Rightarrow \epsilon$. In ϵ , a's and b's are equal (0), so it is not correct option.

In option (A), S will be replaced by either AC or CB. C will either generate no. of a's more than no. of b's by 1 or no. of b's more than no. of a's by 1. But one more a or one more b can be compensated by $B \rightarrow Bb \mid \epsilon$ or $A \rightarrow aA \mid \epsilon$ respectively. So it may give strings with equal no. of a's and b's. So, it is not a correct option.

In option (D), S will be replaced by either AC or CB. C will always generate equal no. of a's and b's. If we replace S by AC, A will add atleast one extra a. and if we replace S by CB, B will add atleast one extra b. **So this grammar will never generate equal no. of a's and b's. So, option (D) is correct.**

References:

1. J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages and Computation*, Second Edition, Addison Wesley, 2001
2. T.A. Sudkamp, *An Introduction to the Theory of Computer Science Languages and Machines*, Third Edition, Addison Wesley, 2006
3. Closure Properties CFL.pdf by Prof. Preet Kanwal, PES University.
4. <https://www.geeksforgeeks.org/closure-properties-table-in-toc/>



Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS242A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Pumping Lemma for Context-Free Languages

Non-Context Free Languages

Question:

Are there any languages that are not context-free? What about $L = \{ a^n b^n c^n \mid n \geq 0 \}$?
(i.e., think of languages for which a stack will not be enough)

Answer:

L is not context-free, because

- Recognizing if $w \in L$ requires remembering the number of a's seen, b's seen and c's seen
- We can remember one of them on the stack (say a's), and compare them to another (say b's) by popping, but not to both b's and c's

The precise way to capture this intuition is through the pumping lemma.

The pumping lemma gives us a technique to show that certain languages are not context free.

Pumping Lemma for CFLs

Theorem: If L is a context free language, then there exists a positive integer n (called a "pumping length") such that for every long string z in L, where $|z| \geq n$, we can divide z into five substrings, $z = uvwxy$, such that

1. $|vwx| \leq n$
2. $|vx| \geq 1$
3. For every $i \geq 0$, the string $uv^iwx^iy \in L$ (In simple words, if strings v and x are 'pumped', simultaneously any number of times, the resultant string still remains in L)

Exercise: Can the following strings be empty?

1) u

3) w

5) y

2) v

4) x

Note: v and x can't both be empty.

Note:

- Pumping Lemma for CFL states that, for all sufficiently long strings $z \in L$, it is possible to find two short, nearby substrings, that can be simultaneously pumped to obtain more words in L.
- Pumping Lemma for CFL states that, if there is a string long enough to cause a cycle, then some variable appears more than once in the derivation of that string and we can always find two pieces of this sufficiently long string to "pump" in tandem and discover an infinite sequence of strings that had to be in the language.

Yield of a CNF Parse Tree (Yield size)

Theorem: Let a parse tree be according to a Chomsky-Normal-Form grammar, and the yield of the tree is z (i.e., z is derived string). If the length of the longest path is N , then $|z| \leq 2^{N-1}$ (Note: N in above theorem also represents height of the parse tree)

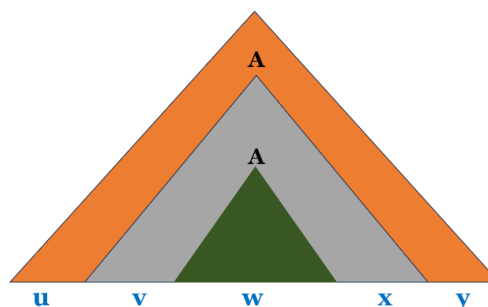
Problem: Prove the above theorem via an induction on N .

Solution:

- **Base Case:** $N = 1$
 - If the longest path is of length 1, we must be using the rule $A \rightarrow t$, so $|z|$ is 1 and $2^{1-1} = 1$
- **Induction**
 - **Longest path has length N** , where $N > 1$. The root uses a production that must be of the form $A \rightarrow BC$ since we can't have a terminal from the root
 - By induction, the subtrees from B and C have yields of length at most 2^{N-2} since we used one of the edges from the root to these subtrees
 - **The yield of the entire tree** is the concatenation of these two yields, which is $2^{N-2} + 2^{N-2}$ which equals $2 * 2^{N-2} = 2^{N-2+1} = 2^{N-1}$

Proof of the Pumping Lemma for CFLs

- Start with a CNF grammar G for Context free language L
- Let the grammar have m non-terminals (or variables).
- Pick pumping constant $n = 2^m$
- Choose a sufficiently long string z in L , where $|z| \geq n$.
Now we know that the parse tree for z has a path with at least $m+1$ variables.
- Consider some longest path.
- There are only m different variables in grammar G , so among the $m+1$ variables in some longest path, we can find two nodes with the same label, say A (that derives the same).
- In the longest path, the variable say A appeared twice, one A represents smaller subtree and the other A represents bigger subtree.
 1. A substring that is derived from smaller subtree of A is w and
 2. The left substring that is derived from bigger subtree of A leaving substring w is v and
 3. The right substring that is derived from bigger subtree of A leaving substring w is x and
 4. The left substring that is derived from outer subtree leaving v, w and x is u and
 5. The right substring that is derived from outer subtree leaving v, w and x is y .

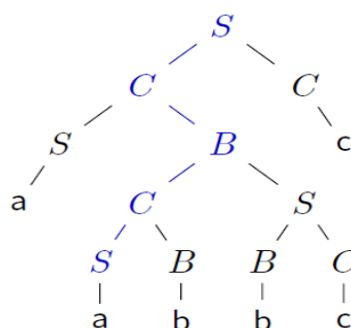


Example: Consider a parse tree for word $z = \text{aabbcc}$ and CFG G in CNF.

$S \rightarrow CC \mid BC \mid a$

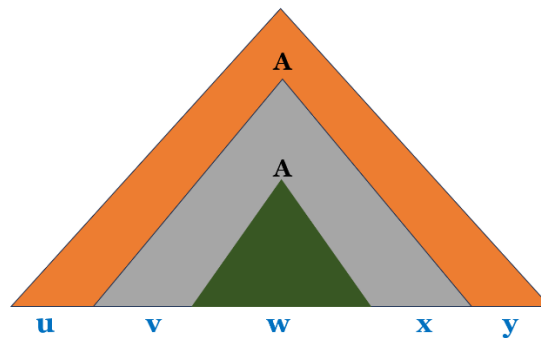
$B \rightarrow CS \mid b$

$C \rightarrow SB \mid c$

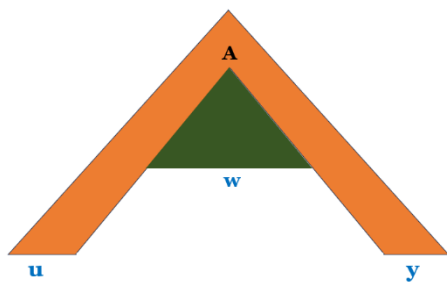


Note: If parse tree is large enough, we will repeat some nonterminal in a path.

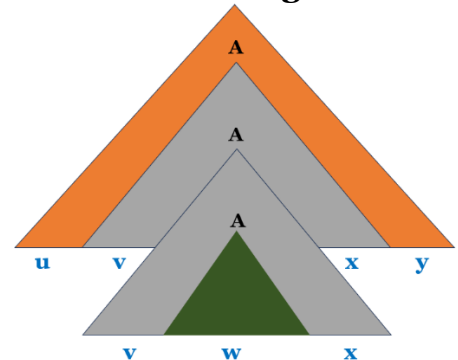
In general, the parse tree thus looks like:



Pump Zero Times the substrings v and x:



Pump Twice the substrings v and x:



Example: Consider CFG in CNF:

$S \rightarrow CC \mid BC \mid a$

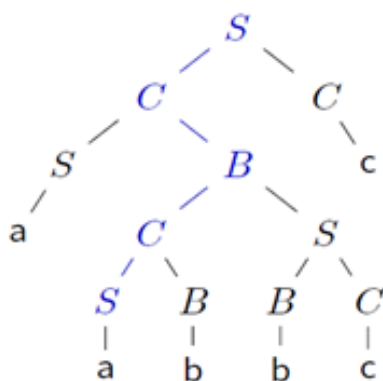
$B \rightarrow CS \mid b$

$C \rightarrow SB \mid c$

Derive a long string: *aabbcc*

$S \Rightarrow CC$ use $C \rightarrow SB$
 $\Rightarrow SBC$ use $S \rightarrow a$
 $\Rightarrow aBC$ use $B \rightarrow CS$
 $\Rightarrow aCSC$ use $C \rightarrow SB$
 $\Rightarrow aSBSC$ use $S \rightarrow a$
 $\Rightarrow aaBSC$ use $B \rightarrow b$
 $\Rightarrow aabSC$ use $S \rightarrow BC$
 $\Rightarrow aabBCC$ use $B \rightarrow b$
 $\Rightarrow aabbCC$ use $C \rightarrow c$
 $\Rightarrow aabbcC$ use $C \rightarrow c$
 $\Rightarrow \mathbf{aabbcc}$

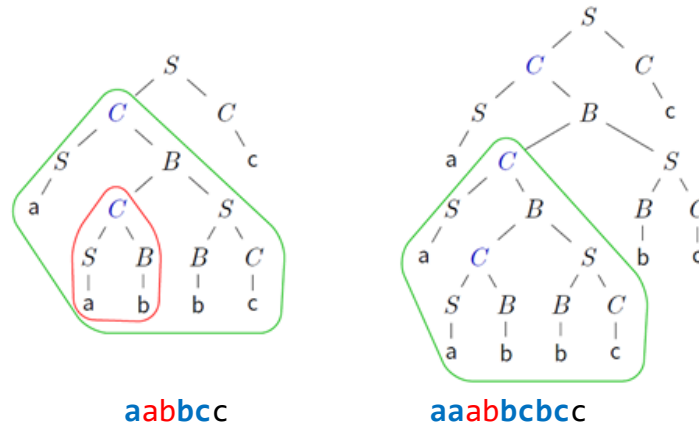
If a derivation is long enough, some variable (that derives same) must appear twice on the same path in a parse tree.



In the path **SCBCSa**, the variable C appeared twice, one C represents smaller subtree and the other C represents bigger subtree.

- **A substring that is derived from smaller subtree of C is w (i.e., $w=ab$) and**
- **the left substring that is derived from bigger subtree of C leaving substring w is v (i.e., $v=a$) and**
- **the right substring that is derived from bigger subtree of C leaving substring w is x (i.e., $x=bc$) and**
- the substring **u** is ϵ and
- the substring **y** is c .

Then we can **cut and paste** part of parse tree, i.e., replace smaller subtree of C by bigger subtree of C.

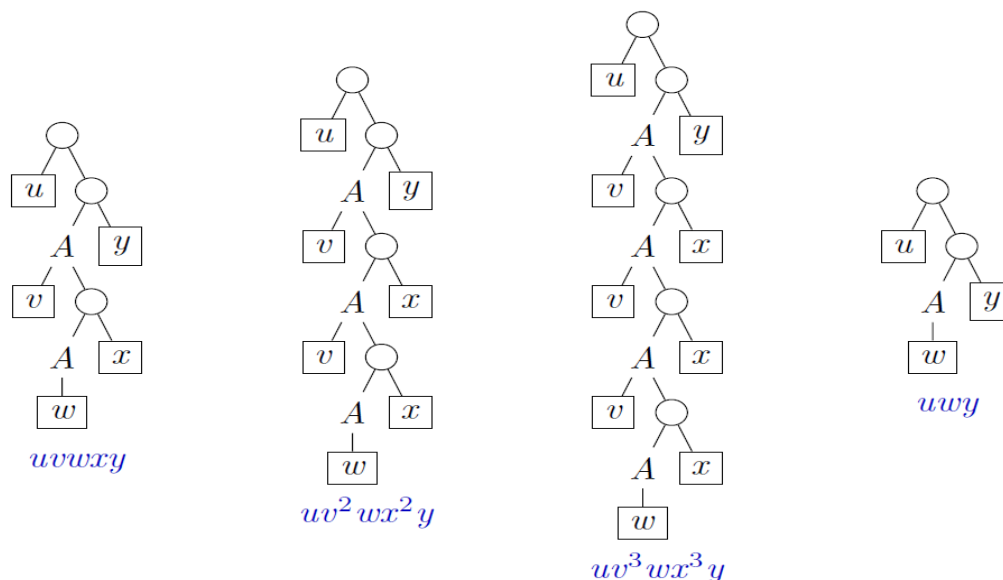


We can repeat this many times

$$aabbcc \Rightarrow aaabbcbbcc \Rightarrow aaaabbcbbcbcc \Rightarrow \dots \Rightarrow (a)^k ab(bc)^k c$$

where $u = \varepsilon$, $v = a$, $w = ab$, $x = bc$, and $y = c$

Pumping in general



The steps to prove that the language is not CFL are:

- Assume that L is context free.
- The pumping length is n .
- All strings longer than n can be pumped $|z| \geq n$.
- Now find a string ' z ' in L such that $|z| \geq n$.
- Divide w into $uvwxy$
- Show that $uv^iwx^iy \notin L$ for some i
- Then, consider the other ways that z can be divided into $uvwxy$.

- Show that none of these can satisfy all the 3 pumping conditions at same time.
- This contradicts the claim made; hence the language L is not context free.

Example 1: Let $L = \{ a^p b^p c^p \mid p \geq 0 \}$. Show that L is not context-free.

Solution:

Since $z = uvwxy$ and $|vwx| \leq n$, we know that the string vwx must consist of either: all or some a's, all or some b's, all or some c's, a combination of a's and b's, a combination of b's and c's

Case 1: (v, w and x containing only a's)

Assume L is context free. Let n be the pumping length.

Choose string z, where $|z| \geq n$.

Let $z = a^n b^n c^n$ where $|z| = 3n$ and $3n \geq n$

Divide $z = uvwxy$ where $|vwx| \leq n$ and $|vx| \geq 1$

$z = a^{n-3} a a a b^n c^n$ where $u = a^{n-3}$ $v = a$ $w = a$ $x = a$ $y = b^n c^n$

Choose $i=2$ (i.e., pumping up v and x by 2)

$z = a^{n-3} a^2 a a^2 b^n c^n$

$z = a^{n+2} b^n c^n \notin L$

Since the **Number of 'a's' != Number of 'b's'**, but the original language has an equal number of a's, b's and c's; therefore, this string will not lie in our language.

If anyone string does not satisfy its conditions, then the language is not CFL.

Case 2: (v, w and x containing only b's)

...

Case 3: (v, w and x containing only c's)

...

Case 4: (v, w and x containing some a's and some b's)

...

Case 5: (v, w and x containing some b's and some c's)

...

Example 2: Let $L = \{ ww \mid w \in \{a, b\}^* \}$. Show that L is not context-free.

Solution:

Assume L is context free. Let n be the pumping length.

Choose string z, where $|z| \geq n$.

Let $z = a^n b a^n b$ where $|z| = 2n+2$ and $2n+2 \geq n$

Divide $z = uvwxy$ where $|vwx| \leq n$ and $|vx| \geq 1$

$z = a^{n-3} a a a b a^n b$ where $u = a^{n-3}$, $v = a$, $w = a$, $x = a$, and $y = b a^n b$

Choose $i=2$ (i.e., pumping up v and x by 2)

$z = a^{n-3} a^2 a a^2 b a^n b$

$z = a^{n+2} b a^n b \notin L$

Since the **Number of a's in first w != Number of a's in second w**, but the original language has an equal number of a's in both the w's, therefore, this string will not lie in our language.

If anyone string does not satisfy its conditions, then the language is not CFL.

Example 3: Let $L = \{ a^k \mid k \in \mathbb{N} \text{ and } k \text{ is prime} \}$. Show that L is not context-free.

Solution:

Assume L is context free. Let n be the pumping length.

Pick a prime number p (that is greater than or equal to pumping constant n)

Let $z = a^p$ with $|a^p| \geq n$ i.e., $p \geq n$

Divide $z = uvwxy$ with $|vwx| \leq n$ and $|vx| \geq 1$.

$z = a^{p-k} a^k$ where $u=a^{p-k}$, $v=a^k$, $w=\epsilon$, $x=\epsilon$, and $y=\epsilon$

Let $|vx| \geq 1$. Then for each $i \in \mathbb{N}$,

$$z = uv^iwx^iy = a^{p-k} (a^k)^i = a^{p-k} a^{ik} = a^{p+(i-1)k}$$

we have $uv^iwx^iy = a^{p+(i-1)k}$, and since these strings are all in L , it must be the case that $p+(i-1)k$ is prime for all $i \in \mathbb{N}$.

Pick $i = 2$. Then the number $p+(i-1)k$ is equal to $p+k$

Pick $i = 3$. Then the number $p+(i-1)k$ is equal to $p+2k$

...

Pick $i = p + 1$. Then the number $p+(i-1)k$ is equal to $p+pk = p(k+1)$.

The product p times $k+1$ is a factorization of the number $p+(i-1)k$ when $i=p+1$, so this number is not prime.

This is a contradiction, so L is not context-free.

Example 4: Let $L = \{ a^{p!} \mid p \geq 0 \}$. Show that L is not context-free.

Solution:

Assume L is context free. Let n be the pumping length.

Choose string z , where $|z| \geq n$.

Let $z = a^{n!}$ where $|z|=n!$ and $n! \geq n$

Divide $z = uvwxy$ where $|vwx| \leq n$ and $|vx| \geq 1$

$z = a^{n!-\beta} a^\beta$ where $u=a^{n!-\beta}$, $v=a^\beta$, $w=\epsilon$, $x=\epsilon$, and $y=\epsilon$

$z = a^{n!-\beta} a^\beta$ where $|vwx| \leq n$ i.e., $|a^\beta| \leq n$ i.e., $\beta \leq n$
and $|vx| \geq 1$ i.e., $|a^\beta| \geq 1$ i.e., $\beta \geq 1$

$$z = uv^iwx^iy$$

$$z = a^{n!-\beta} (a^\beta)^i$$

$$z = a^{n!-\beta+\beta i}$$

$$z = a^{n!+(i-1)\beta}$$

If $z \in L$, $n!+(i-1)\beta$ is a factorial

Choose $i=2$ (i.e., pumping up v by 2)

$$z = a^{n!+(2-1)\beta} = a^{n!+\beta} \Rightarrow n!+\beta \text{ is a factorial}$$

$$n! < (n!+\beta) \quad \text{because } \beta \geq 1$$

$$(n!+\beta) \leq (n!+n) \quad \text{because } \beta \leq n$$

$$n! < (n!+\beta) \leq (n!+n) \quad [\text{we know that } (n+1)! \text{ is } (n+1)*n! \text{ and hence } (n!+n) < (n+1)!]$$

$$z = a^{n!+\beta} \notin L$$

If any string does not satisfy its conditions, then the language is not CFL.

Exercises:

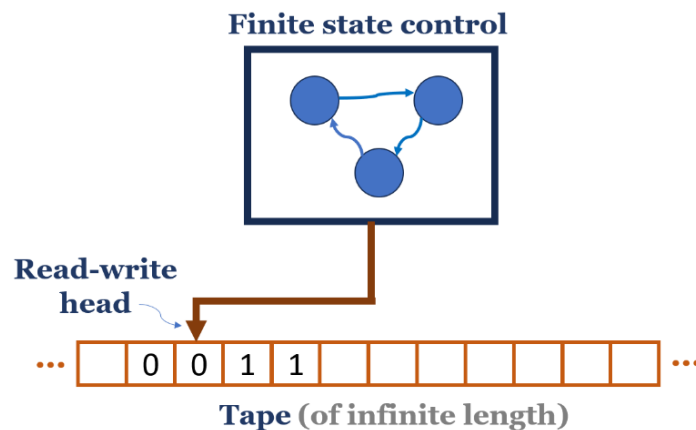
1. Let $L = \{ a^n b^m a^n b^m \mid n, m \in \mathbb{N} \}$. Show that L is not context-free.
2. Let $L = \{ a^p \mid p \in \mathbb{N} \text{ and } p \text{ is } n^2 \}$. Show that L is not context-free.
3. Let $L = \{ a^n b^m c^k \mid n, m, k \in \mathbb{N} \text{ and } k=n*m \}$. Show that L is not context-free.
4. Let $L = \{ r\#s \mid r, s \in \{0, 1\}^*, r \text{ is a substring of } s \}$. Show that L is not context-free.

References:

1. <http://infolab.stanford.edu/~ullman/ialc/spr10/slides/cfl4.pdf>
2. <https://web.iitd.ac.in/~bspanda/pumpinglemmacfl.pdf>
3. <https://web.cs.hacettepe.edu.tr/~ilyas/Courses/BBM401/lec08-CFLProperties.pdf>
4. <https://www.cse.cuhk.edu.hk/~siuon/csci3130-f17/slides/lec12.pdf>

Turing Machine: The Foundation of Modern Computing

- A Turing Machine is a theoretical computational model equivalent to a digital computer.
 - It was invented by the mathematician Alan Turing in 1936, and has been since then the most widely used model of computation in computability and complexity theory.
 - Despite its simplicity, the Turing machine can simulate ANY computer algorithm, no matter how complicated it is!
-



Note: By convention, the input for computation is preloaded on the tape and if required special markers can be used to indicate the start and end of the input string.

The Turing Machine comprises of three main components:

1. **The Tape** (of infinite length),
2. **The Read-Write head**, and
3. **Finite state control** (i.e., FSM).

The Tape:

- Turing machine consists of an infinitely-long memory tape which acts like the memory in a typical computer, or any other form of data storage.
- The tape is divided into squares(discrete cells), and each of which can hold a single symbol drawn from a finite set of symbols called the alphabet of the machine.
- A Turing machine can read, write, and erase symbols on an infinitely long tape.

The read-write head:

- The head is a device that can read, write and erase symbols on the cell individually and move left or right along the tape one cell at a time.
- Turing machine head can only read/write one symbol at a time, and the choice of which direction to move the head, and whether to halt is based on a finite table that specifies what to do for each combination of the current state and the symbol that is read.

Note:

- Like a real computer program, it is possible for a Turing machine to go into an infinite loop which will never halt.
- The problem with Turing Machines is that a different one must be constructed for every new computation to be performed, for every input output relation.
- In the context of automata theory and the theory of computation, **Turing machines are used to study the properties of algorithms and to determine what problems can and cannot be solved by computers**. They provide a way to model the behavior of algorithms and to analyze their computational complexity, which is the amount of time and memory they require to solve a problem.
- Alan Turing's groundbreaking work on the theoretical foundations of computation paved the way for the development of modern computing technology.

Formal Definition of Turing Machines

Formally, a deterministic one-tape Turing machine is a 7-tuple

$$M = (Q, \Gamma, \square, \Sigma, \delta, q_0, F)$$

where:

- Q is a finite non-empty set of states
- Γ is a finite non-empty set of tape symbols (which can be written on Tape)
- $\square$ is a blank symbol (every cell is filled with blank except input alphabet initially), and $\square \in \Gamma$
- Σ is the input alphabet (symbols which are part of input alphabet), and $\Sigma \subseteq \Gamma$
- δ is a transition function which maps $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$.

Depending on its **present state** and **present tape alphabet** (pointed by read-write head), it will move to **new state**, **change the tape symbol** (may or may not) and **move head to either left or right**.

- q_0 is the initial (or start) state
- F is the set of final (or accept) states. If any state of F is reached, input string is accepted.

Variants of a Turing Machine

- Multi-tape TMs.
- Non-deterministic TMs
- Multi-head TMs
- Single sided vs double sided infinite tape TMs
- Turing Machines with a Stay-Option ($\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$)
- Turing machine with multiple tracks
- ...

Non-deterministic TMs

At any point in the computation, the TM may proceed according to several possibilities. Thus, the transition function has the form $\delta: Q \times \Gamma \rightarrow 2^Q \times \Gamma \times \{L, R\}$

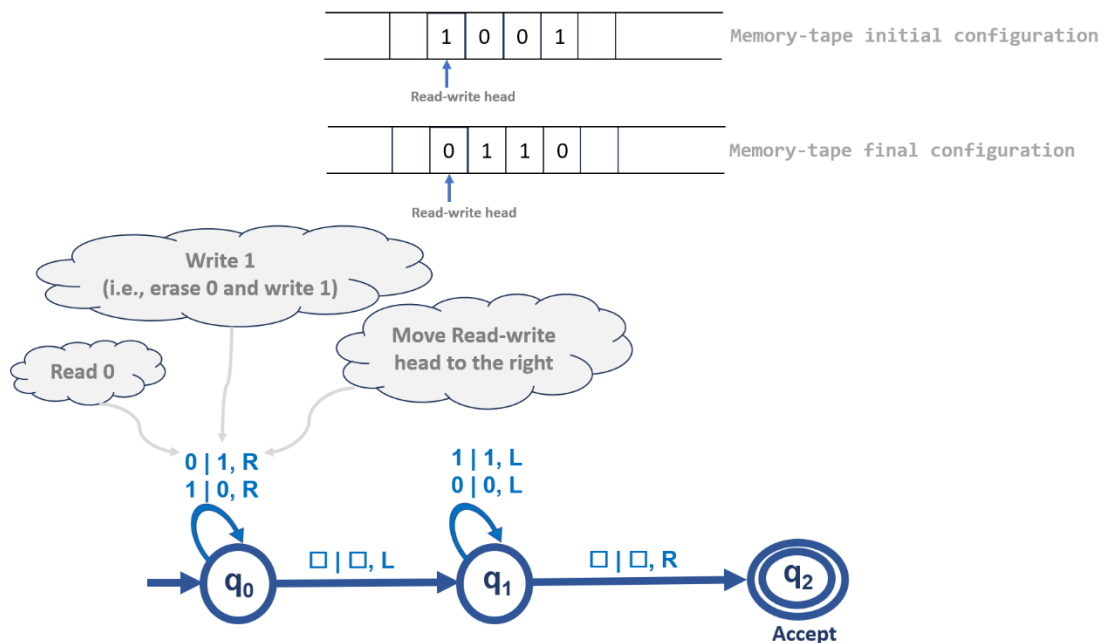
Deterministic TMs Examples:

1) Construct a Turing Machine as Transducer to convert a given binary number to its 1's complement and move the head back to the first symbol after complement.

Solution: High-Level Idea (or Logic)

1. Sweep from left to right, Turn zero to one and one to zero.
2. Move the head back to the first symbol after complement.

Transition Diagram:



Transition function δ is given in Table as:

δ	0	1	$\square$
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	$(q_1, \square, L)$
q_1	$(q_1, 0, L)$	$(q_1, 1, L)$	$(q_2, \square, R)$
q_2^*	-	-	-

Note:

- Accepting states in Turing machine have no outgoing transitions.
- Other accepted transition notations: $0 \rightarrow 1, L$ or $0, 1, L$ or $0|1|L$ or $0,1|L$

2) Construct a Turing Machine as decider for $L = \{ a^n b^n c^n \mid n \geq 1 \}$

Solution:

We want to accept, if and only if:

- the input string contains equal number of a's followed by equal number of b's followed by equal number of c's.

Pseudocode:

1. Replace a by x, scan to the right until b.
2. Replace b by y, scan to the right until c.
3. Replace c by z, scan to the left until x.

4. If there's another 'a' left, then repeat step 1 to 3.

5. If a's are over scan to the right and

Check if all a's are replaced by x, and

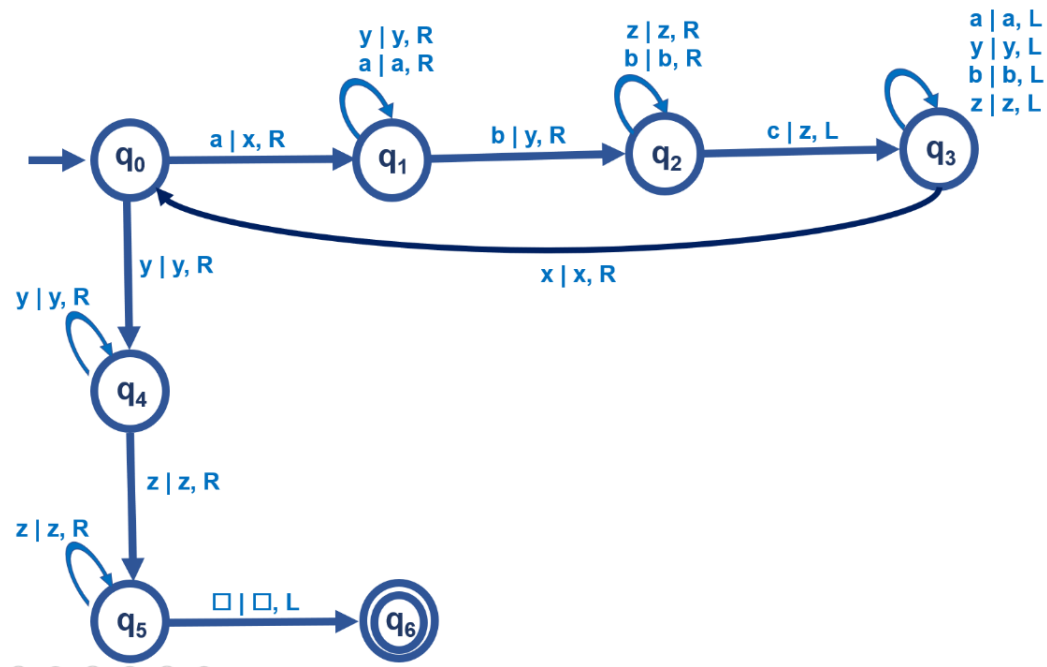
Check if all b's are replaced by y, and

Check if all c's are replaced by z.

If yes, then *accept*, else *reject*.

(i.e., if a's are over scan to the right (seeing only x,y and z) until blank ($\square$) symbol and accept)

Note: For more a's q_1 is a trap state, For more b's q_4 is a trap state, For more c's q_5 is a trap state,



3) Construct a Turing Machine as decider for $L = \{ 0^{2^n} \mid n \geq 0 \}$

Solution:

We want to accept, if and only if:

- the input string consists entirely of zeros, and
- the number of zeros is a 2 power (i.e., 1, 2, 4, 8, 16, 32, ...)

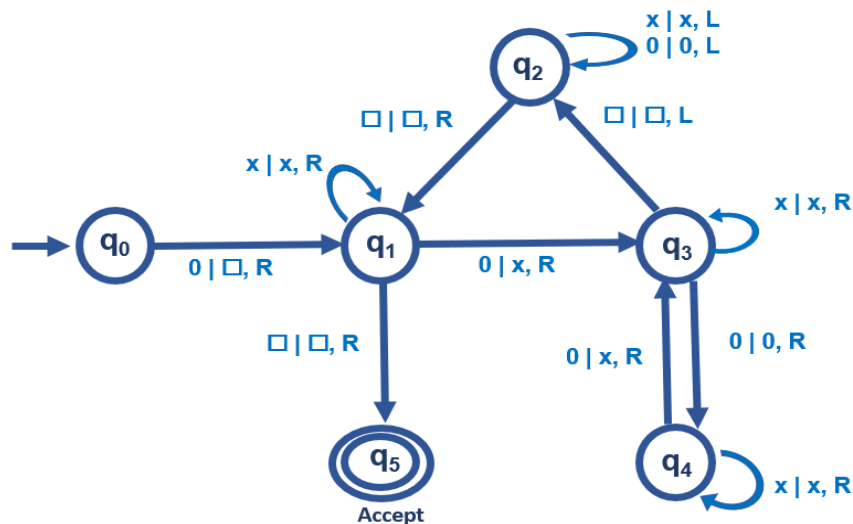
High-Level Idea (or Logic).

- Repeatedly divide the number of zeros in half (2 equal parts of zeros) until it becomes an odd number.
- If we are left with a single zero, then accept, Otherwise, reject.

Pseudocode:

- Turn first zero to blank.
- Sweep from left to right, cross out every other zero (i.e., alternate zero). (This divides number of zeros into half, including the first zero i.e., turned into blank)
 - If the tape had only one 0 (including the first zero i.e., turned into blank), *accept*.
 - Else **if further equal division is not possible** and **the tape had an odd number of 0's** (including the first zero i.e., turned into blank), *reject*.
- Move the head back to the first input symbol. Repeat step-2 process

Transition Diagram:



Note: The Turing machine halts in a state if there is no transition to follow.

This is a state diagram corresponding to the pseudocode.

- Let's think about how the two correspond.
- We have an extra symbol "x" that is used to cross out symbols.

We start by writing the first 0 over with a blank.

- If there was just a single 0, we accept.
- In states q3 and q4, we cross out every other 0, until we see a blank.
- If we see a blank in state q4, then the number of 0's left was odd, so we reject.
- If we see a blank in state q3, then the number was even, so now in q2 we move left until we hit blank. We move back to q1

4) Construct a Turing Machine as decider for $L = \{a^n b^m c^{n^*m} \mid n, m \geq 1\}$

Solution:

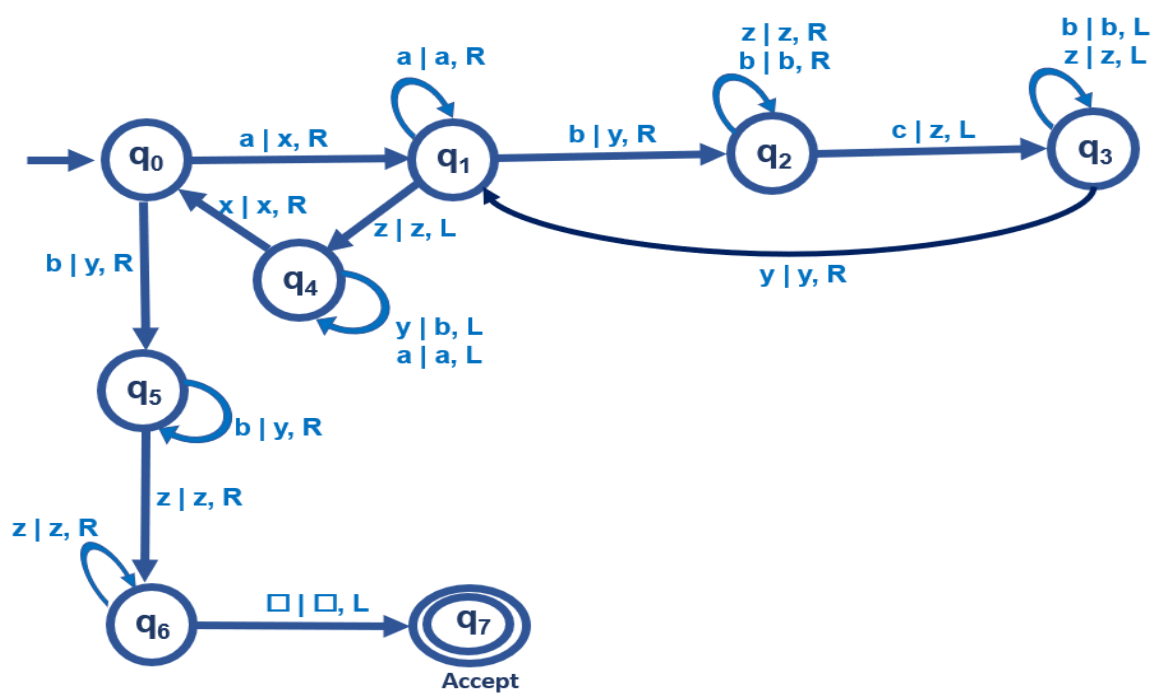
High-Level Idea (or Logic).

```
For each occurrence of a: {  
  For each occurrence of b: {  
    Replace an occurrence of c by z.  
  }  
}
```

Pseudocode:

1. Replace a by x, scan to the right until b.
2. Replace b by y, scan to the right until c.
Replace c by z, scan to the left until y. goto step 2
If too few c's, reject.
3. Replace all y's by b.
4. If there's another 'a' left, then repeat from step 1.
If all a's are replaced by x,
Check if all c's are replaced by z.
If yes, then *accept*, else *reject*.

Transition Diagram:



☐ aabbbbceeecc
☐ xabbbbceeecc
☐ xaybbbceeecc
☐ xaybbzceeecc
☐ xayybzceeecc
☐ xayyzceeecc
☐ xayyyzceeecc

☐ xxbbbbzceec
☐ xxybbzceec
☐ xxybbzceec
☐ xxyybzceec
☐ xxyybzceec
☐ xxyyyzceec

☐ xxyybzceec
☐ xxyybbzceec
☐ xxbbbbzceec

☐ xxyybbzceec
☐ xxyybzceec
☐ xxyyyzceec

5) Construct a Turing Machine as Transducer for the following. Given an input: $0^m 1 0^n$, write 0^{m*n} as output on the tape (Language of Unary number multiplication).

Input and output must be separated by a blank cell (i.e., $\square$).

Solution:

High-Level Idea (or Logic).

For each occurrence of 0 before 1: {

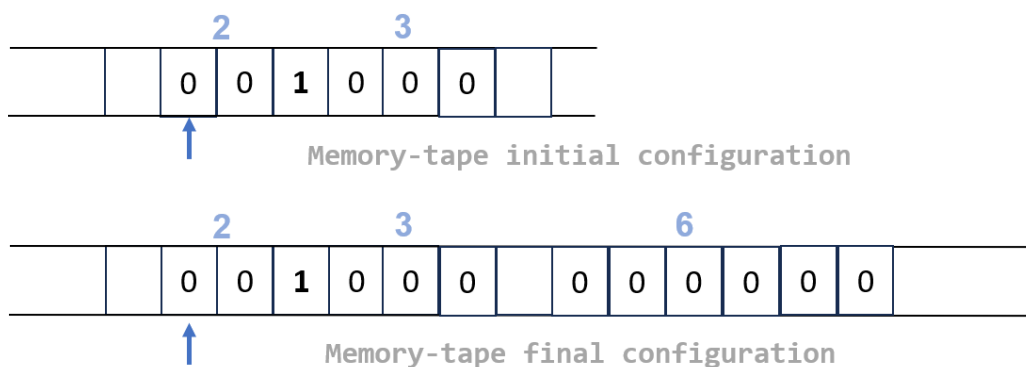
For each occurrence of 0 after 1: {

Write(after input leaving one blank) a digit 0 on tape i.e., replace blank by 0.

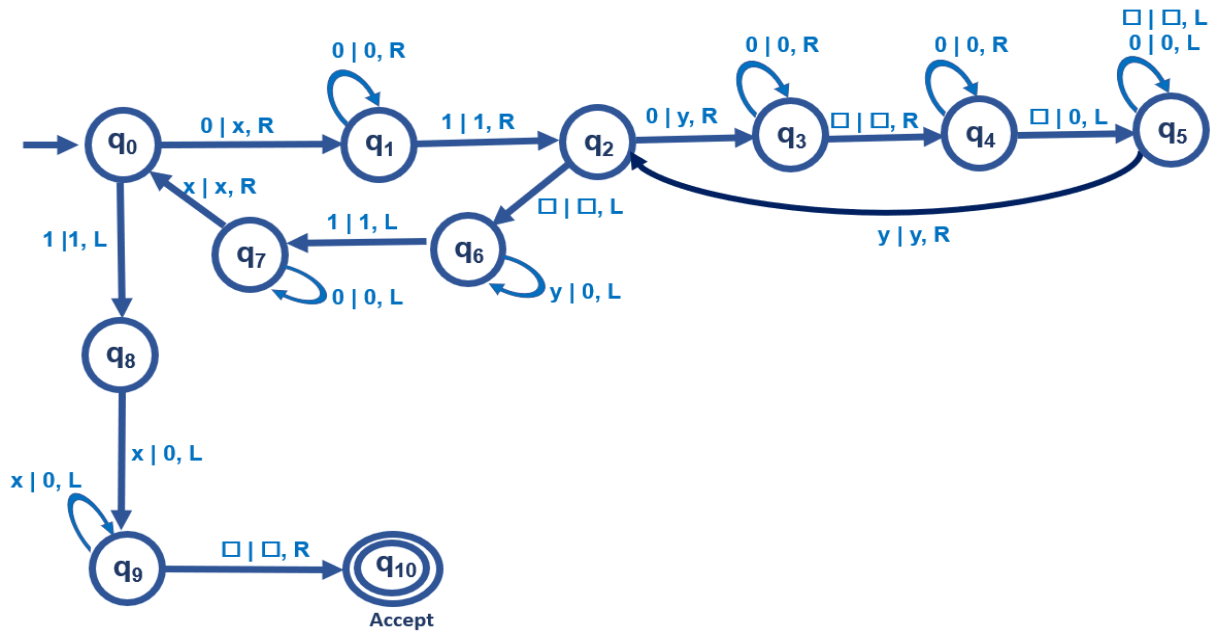
$$\}$$

}

Memory-tape initial and final configuration.

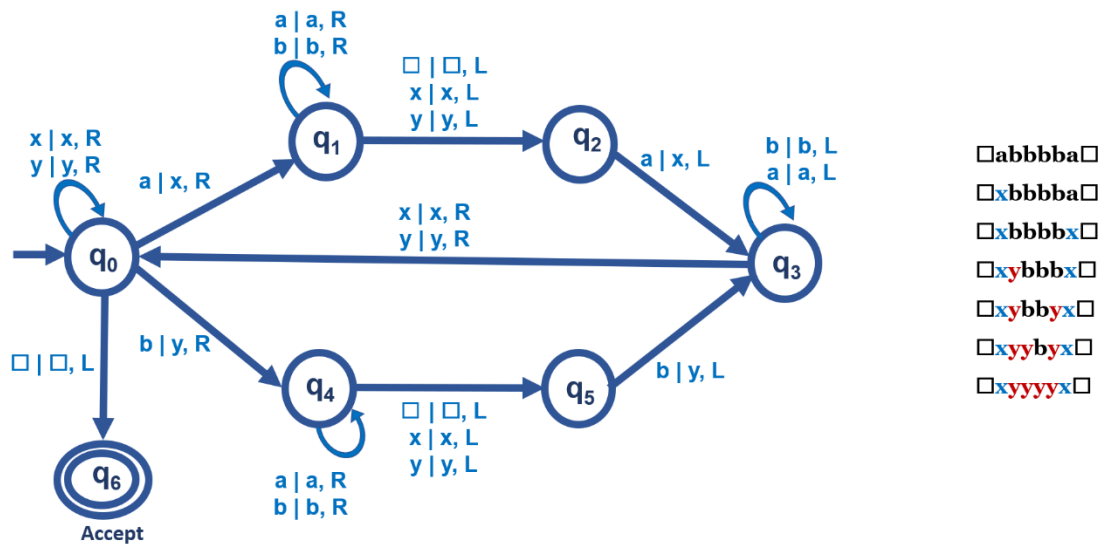


Turing machine diagram



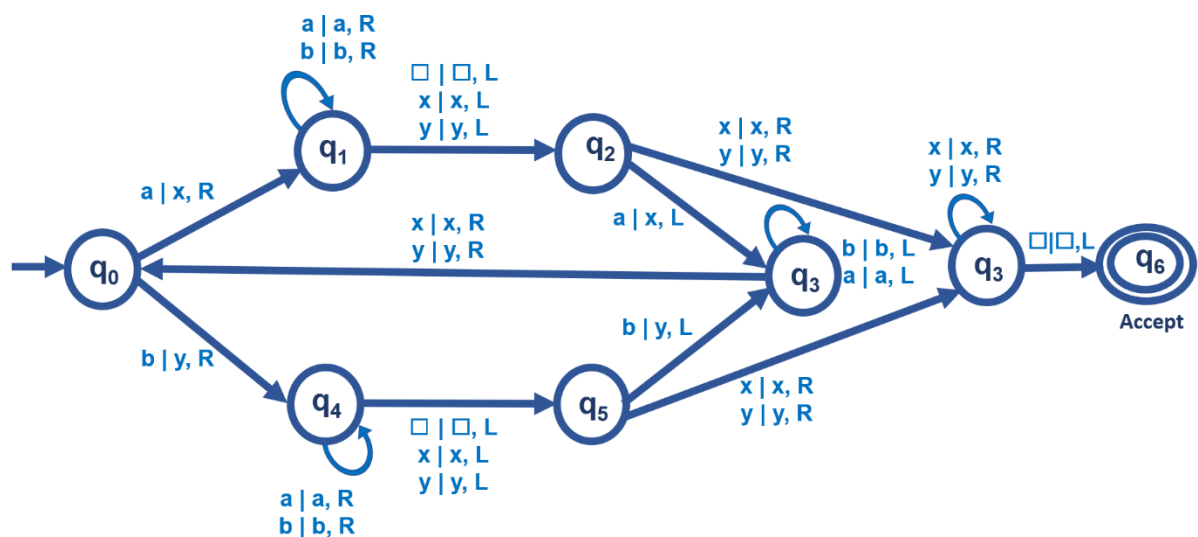
6) Construct a Turing Machine as decider for language $L = \{ww^R \mid w \in \{a, b\}^*\}$

Solution:



7) Construct a Turing Machine as decider for language $L = \{waw^R \text{ or } wbw^R \mid w \in \{a, b\}^*\}$

Solution:



Input Acceptance in TM:

- Turing machine M accepts an input string, if M halts in an accept state.

Input Rejection in TM:

- If Turing machine M halts in a non-accepting state, then it rejects the input string.
- If Turing machine M enters an infinite-loop (runs forever), then also it rejects the input string.

Types of Halting in TM:

- **Halting in an accept state**, then accept the input string.
- **Halting in a non-accept state**, then reject the input string.

Exercises:

1) Construct a Turing Machine as Transducer for the following

1. Given input: $0^m + 0^n$, write 0^{m+n} as output on the tape (**Language of addition in unary number system**), $m, n \geq 1$
2. Given a string $w \in \{a,b\}^+$, write w as output on the tape separated by a blank. Final contents on the tape must look like: $w \square w$ (**Copy operation**)
3. Given an input: $a^n b^m$, Write the difference $k = n - m$ (in terms of no. of c's) on the tape after input. If the difference is negative append a negative sign.
For example: If $k = 3$ write ccc on the tape.
If $k = -2$, write -cc on the tape. (Language of subtraction).
Input and output must be separated by a $\square$.
4. Given a string $w \in \{a,b\}^+$, **sort the symbols in the input string**. For example, if the input is babaa, output should be: aaabb. Final contents on the tape must look like: **babaa $\square$ aaabb**
5. Given a string $w \in \{a,b\}^+$, write w^R as the output on the tape (Reversing a string). Final contents on the tape must look like : $w \square w^R$

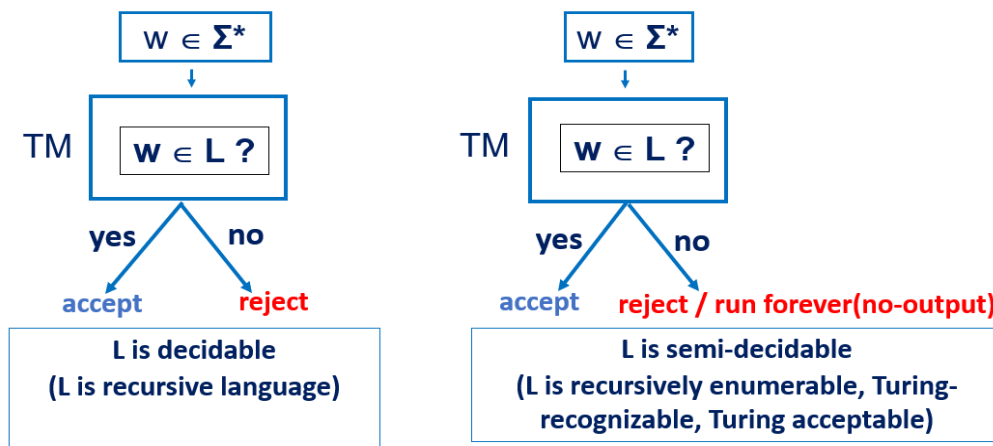
2) Construct a Turing Machine which compare two numbers ($A < B$, $A = B$, $A > B$). Using unary number format to represent the number.

For example, 4 is represented by 1 1 1 1 or 0 0 0 0

3) Construct a Turing Machine as decider for the following languages

1. $L = \{ 0^n 1^{n^2}, n \geq 1 \}$
2. $L = \{ ww, w \in \{a,b\}^+ \}$
3. $L = \{ w \# w, w \in \{0,1\}^* \}$
4. $L = \{ a^n b^m c^{nd^m}, n \geq 1 \}$
5. L is a given regular language, say $L = \{ (a + b)^* b (a + b)^* \}$.
 - a. Tip: Simulate an DFA using a TM?
6. L is a given context-free language, say $L = \{ w : w \text{ is a palindrome} \}$
 - a. Tip: Simulate an PDA using a TM?
7. $L = \{ a^p \mid p \text{ is a prime number} \}$

Turing-Decidable and Turing-Recognizable



Theorem: L is decidable, if both L and L^c are recursively enumerable

Turing-Decidable:

- Turing Machines that halt on all inputs (i.e., it never loops).
- The language recognized by a Turing Machine Decider is called **Recursive Language**.
- A language for which there exists a Turing Machine that can determine if a given string is part of the language or not in a finite amount of time.

The recursive languages

- The set of all languages that are decided by some Turing machine.
- All languages described by a non-looping TM.

A language is “Turing-Recognizable” iff there exists a Turing Machine such that

- when encountering a string in that language, the machine terminates and accepts that string;
- when encountering a string not in that language, the machine either terminates and rejects that string or doesn't terminate at all.

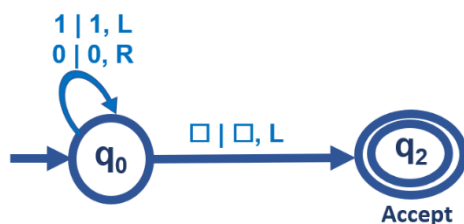
Note: there is no requirement that the Turing Machine should halt for strings not in the language.

A language is “Turing-Decidable” iff there exists a Turing Machine such that

- when encountering a string in that language, the machine terminates and accepts that string;
- when encountering a string not in that language, the machine terminates and rejects that string.

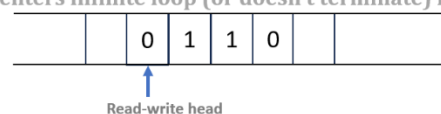
Note: A language that is “Turing-Decidable” is a stronger condition than “Turing-Recognizable”, because, if a language is Turing-Decidable then its corresponding Turing Machine never runs forever.

Note: Example for machine entering Infinite Loop or doesn't terminate for some inputs.

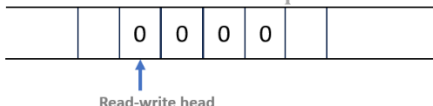


This machine accepts and halts for strings made up of only 0's or strings begins with 1.

TM enters infinite loop (or doesn't terminate) for this input.



TM terminate for this input.



Encoding Turing machines as strings

- Every Turing machine can be encoded as a string in $\{0, 1\}^*$, i.e., Just encode the description of the machine (in binary)!
- Every string in $\{0, 1\}^*$ is a TM
- Of course, some strings don't represent a valid encoding, then we can map them to a TM that does nothing.

Notation

$\langle M \rangle$, a string representation of Machine M Description.

Universal Turing Machine (U_{TM})

- In his 1936 paper "On Computable Numbers", Turing proved that **we can build a Turing machine U_{TM} that acts as an interpreter for other Turing machines.**
In other words, U_{TM} 's input tape can contain a description of another Turing machine, which is then simulated step by step. Such a machine U_{TM} is called a Universal Turing machine.
- If a universal machine didn't exist, then in general we would need to build new hardware every time we wanted to solve a new problem: there wouldn't even be the concept of software.

U_{TM} – Universal Turing machine (has multiple tapes: Input, output, working, ...)

$(\langle M \rangle, w)$ – Description of Turing Machine-M encoded in binary and an input string w.



U_{TM} **simulate the behavior of Turing Machine M and runs M on input w,**

- U_{TM} accepts $(\langle M \rangle, w)$, if M accepts w.
- U_{TM} rejects $(\langle M \rangle, w)$, if M rejects w.
- U_{TM} runs for ever on $(\langle M \rangle, w)$, if M runs for ever on w.

In simple terms, a **Universal Turing Machine is like a "computer" that can simulate the behavior of any other Turing Machine.**

- It achieves this by reading the description of another Turing Machine from its input tape. This **description (encoded in binary) specifies the states and transitions of the other Turing Machine** and enables the Universal Turing Machine to simulate the behaviour of the other machine on its own tape. (i.e., U_{TM} simulates the behaviour of the read Turing machine)
- The Universal Turing Machine then reads the input tape for input w, and uses the simulated Turing Machine to compute the desired output.
- The output is then written onto the output tape, and the Universal Turing Machine stops.

A universal turing machine can thus simulate any other machine. Any problem that is not computable by the universal turing machine is considered to be non-computable.

Note:

- The Universal Turing Machine (U_{TM}) is considered one of the fundamental concepts in computer science and is the theoretical foundation of modern computing.

- U_{TM} demonstrates the power of a single, general-purpose computing device and is a testament to the concept of algorithmic computation.
- A Universal Turing Machine is similar to a regular Turing Machine but it has solutions to all problems that are computable.
- Complex Turing machines can be constructed by combining Turing Machines that perform simple tasks.

Questions on Universal Turing Machine

1. What can a Turing machine compute?

Solution: A Turing machine can theoretically compute anything that is computable. It can simulate the behavior of any algorithm or **solve any problem that has a well-defined algorithmic solution. This property is known as Turing completeness.**

2. What is the Halting Problem, and why is it important in Turing machine theory?

Solution: The Halting Problem is a fundamental problem in computer science and mathematics. It asks whether, **given a description of a Turing machine and its input, can we determine whether the machine will eventually halt (terminate) or run forever on that input.**

Alan Turing proved that the Halting Problem is undecidable.

Turing's efforts were instrumental in proving that **a general algorithm that solves the Halting problem for all possible program-input pairs cannot exist.** As such, he demonstrated the restrictions of computers, thus establishing a limitation on the power of mechanical computation.

3. Can a Turing machine simulate any modern computer?

Solution: Yes, in theory, a Turing machine can simulate any modern computer, as long as there is enough tape and time available. However, practical considerations, such as the enormous amount of tape needed for even simple computations, make this infeasible for complex computations.

The Church-Turing Thesis (aka computability thesis)

Related to the idea of universal machines is the so-called Church-Turing thesis, which **claims that anything we would naturally regard as "computable" is actually computable by a Turing machine.**

There are various equivalent formulations of the Church-Turing thesis.

1. A common one is that every effective computation can be carried out by a Turing machine.
2. Anything that can be computed by an algorithm can be computed by a Turing Machine.
3. What can be computed using Turing machine is known as computable.
4. For any computable problem, there is a Turing machine which computes it.

The Turing Test

The Turing test, one of the most discussed methods for assessing artificial intelligence (AI).

The Turing test, originally called the imitation game by Alan Turing in 1950, is a test of a machine's ability to exhibit intelligent behaviour equivalent to, or indistinguishable from, that of a human.

The Turing Test is a widely used **measure of a machine's ability to demonstrate human-like intelligence.**

References:

- <https://www.cs.cmu.edu/~emc/flac09/reading.html>
- <https://web.stanford.edu/class/archive/cs/cs103/cs103.1142/lectures/18/Small18.pdf>
- <https://plato.stanford.edu/entries/turing-machine/#:~:text=Turing%20machines%2C%20first%20described%20by,the%20computing%20of%20real%20numbers.>
- <https://www.cl.cam.ac.uk/projects/raspberrypi/tutorials/turing-machine/four.html>
- <https://web.mit.edu/manoli/turing/www/turing.html>
- <https://www.britannica.com/technology/Turing-test>
- <https://introcs.cs.princeton.edu/java/52turing/>

Videos:

- A Turing Machine
 - <https://www.youtube.com/watch?v=eZ13KTaQLts>
- A Turing Machine - Overview
 - <https://www.youtube.com/watch?v=E3keLeMwfHY>
- A Turing Machine - Counting Program
 - <https://www.youtube.com/watch?v=WJ-ODmFjmrU>
- A Turing Machine - Busy Beaver 4-state
 - <https://www.youtube.com/watch?v=2PjU6DJyBpw>
- A Turing Machine - Subtraction
 - <https://www.youtube.com/watch?v=aBToqFJLrl4>
- LEGO Turing Machine
 - <https://www.youtube.com/watch?v=FTSAiF9AHN4>

END

Additional Info:

Formulations of Turing's thesis in terms of numbers

Computable numbers are (real) numbers whose decimal representation can be generated progressively, digit by digit, by a Turing machine.

Examples are:

- any number whose decimal representation consists of a finite number of digits (e.g., 109, 1.142)
- all rational numbers, such as one-third, two-sevenths, etc.
- some irrational real numbers, such as π and e .

Some real numbers, though, are uncomputable, as Turing proved. Turing's proof pointed to specific examples of uncomputable real numbers, but it is easy to see in a general way that there *must* be real numbers that cannot be computed by any Turing machine, since there are *more* real numbers than there are Turing-machine programs. There can be no more Turing-machine programs than there are whole numbers, since the

programs can be counted: 1st program, 2nd program, and so on; but, as Georg Cantor proved in 1874, there are vastly more real numbers than whole numbers (Cantor 1874).

A (real) number is Turing computable if there exists a Turing machine which computes an arbitrarily precise approximation to that number. All of the algebraic numbers (roots of polynomials with algebraic coefficients) and many transcendental mathematical constants, such as e and π are Turing-computable. Turing gave several examples of classes of numbers computable by Turing machines

Note:

A rational number is a number that is expressed as the ratio of two integers, where the denominator should not be equal to zero, whereas an irrational number cannot be expressed in the form of fractions. **Rational numbers are terminating decimals but irrational numbers are non-terminating (endless) and non-recurring digits after the decimal point.** Value of $\sqrt{5} = 2.2360679775\ldots$ It is a non-terminating value and hence cannot be written as a fraction. It is an irrational number.

<https://plato.stanford.edu/entries/church-turing/>
<https://plato.stanford.edu/entries/turing-machine/#CompNumbProb>

How Does the Turing Test Work?

The Turing Test is performed by placing a human in one room and a machine in another. Then a judge, or panel of judges, addresses each room with questions regarding any topic to which a human should be able to respond. If the machine passed Turing's test, it shows the machine's ability to process human syntax and semantics, which is thought to be a step toward creating artificial general intelligence.

Regardless of a computer's ability to pass the Turing Test, there is no real way for us to tell whether or not a machine truly understands human semantics. The test simply judges machines on their ability to converse with human-like eloquence, not human-like understanding. This limitation has led some AI researchers to argue the Turing Test is less relevant than it used to be.

Turing Test Example Questions

While there is no official list of Turing Test questions, a judge would likely ask questions that relate to human experiences like emotions and maturation, or linguistic riddles that could be difficult for a machine to parse. Here are some questions to ask if you find yourself judging a Turing Test:

-
- What is your most memorable childhood event and how has that impacted you today?
 - Describe yourself using only colors and shapes.
 - Describe why time flies like an arrow but fruit flies like a banana?
 - How do you feel when you think about your upbringing and what makes you feel that way?
 - What historical event changed you the most and where were you when it happened?
 - Which of the previous questions was the most difficult to answer and why?

References:

What is a CAPTCHA?

A CAPTCHA test is designed to determine if an online user is really a human and not a [bot](#). CAPTCHA is an acronym that stands for "Completely Automated Public [Turing test](#) to tell Computers and Humans Apart." Users often encounter CAPTCHA and reCAPTCHA tests on the Internet. Such tests are one way of [managing bot activity](#), although the approach has its drawbacks.

Although CAPTCHAs are designed to block automated bots, CAPTCHAs are themselves automated. They're programmed to pop up in certain places on a website, and they automatically pass or fail users.

How does a CAPTCHA work?

Classic CAPTCHAs, which are still in use on some web properties today, involve asking users to identify letters. The letters are distorted so that bots are not likely to be able to identify them. To pass the test, users have to interpret the distorted text, type the correct letters into a form field, and submit the form. If the letters don't match, users are prompted to try again. Such tests are common in login forms, account signup forms, online polls, and e-commerce checkout pages.

Turing Machine:

At any one time, the machine has a head which is positioned over one of the squares on the tape and a "state" selected from a finite set of states. With this head, the machine can perform three very basic operations:

- Read the symbol on the square under the head.
- Edit the symbol by writing a new symbol or erasing it
- Move the tape left or right by one square so that the machine can read and edit the symbol on a neighbouring square.

At each step of its operation, the head reads the symbol in its cell. Then, based on the symbol and the machine's own present state, the machine writes a symbol into the same cell or the machine makes no changes on to the cell, and moves the head one step to the left or the right, or halts the computation.

Turing machine can only read one symbol at a time, and the choice of which replacement symbol to write, which direction to move the head, and whether to halt is based on a finite table that specifies what to do for each combination of the current state and the symbol that is read.

Note:

- Like a real computer program, it is possible for a Turing machine to go into an infinite loop which will never halt.
- The problem with Turing Machines is that a different one must be constructed for every new computation to be performed, for every input output relation.
- Turing Machine forms the foundation of theoretical computer science. Because of its simple description and behavior, it is amenable to mathematical analysis. This analysis has led to a deeper understanding of digital computers and computation, including the revelation that there are some computational problems that cannot be solved on computers at all, no matter how fast the processor, or how much memory is available.
- In the context of automata theory and the theory of computation, **Turing machines are used to study the properties of algorithms and to determine what problems can and cannot be solved**

by computers. They provide a way to model the behavior of algorithms and to analyze their computational complexity, which is the amount of time and memory they require to solve a problem.

- Today, Turing Machines are considered to be one of the foundational models of computability and (theoretical) computer science.
- A Turing machine (automaton) is capable of enumerating some arbitrary subset of valid strings of an alphabet. A set of strings which can be enumerated in this manner is called a Recursively Enumerable language (generated by Type-0 Grammar). The Turing machine can equivalently be defined as a model that recognises valid input strings, rather than enumerating output strings.
- Given a Turing machine M and an arbitrary string s , it is generally not possible to decide whether M will eventually produce s . This is due to the fact that the halting problem is unsolvable, which has major implications for the theoretical limits of computing.
- The Turing machine is capable of processing an unrestricted grammar, which further implies that it is capable of robustly evaluating first-order logic in an infinite number of ways. This is famously demonstrated through lambda calculus.
- Turing machines are widely accepted as a synonym for algorithmic computability (Church-Turing thesis). Using these conceptual machines Turing showed that first-order logic validity problem is non-computable. That is there exists some problems for which you can never write a program no matter how hard you try!
- Alan Turing's groundbreaking work on the theoretical foundations of computation paved the way for the development of modern computing technology.